

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

Desarrollo de un Sistema de Guiado Autónomo y Control de Lazo Cerrado para UAVs en Interiores Industriales

AUTOR: NAÍM RODRÍGUEZ REYES

Puerto Real, junio de 2026

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

Desarrollo de un Sistema de Guiado Autónomo y Control de Lazo Cerrado para UAVs en Interiores Industriales

TUTOR: PEDRO LUIS GALINDO
AUTOR: NAÍM RODRÍGUEZ REYES

Puerto Real, junio de 2026

Declaración personal de autoría

Naím Rodríguez Reyes con DNI 12345678A, estudiante del Grado Universitario en Ingeniería Informática en la Escuela Superior de Ingeniería de la Universidad de Cádiz, como autor de este documento académico titulado 'Desarrollo de un Sistema de Guiado Autónomo y Control de Lazo Cerrado para UAVs en Interiores Industriales' y presentado como Trabajo Final de Grado.

DECLARO QUE

Es un trabajo original, que no copio ni utilizo parte de obra alguna sin mencionar de forma clara y precisa su origen tanto en el cuerpo del texto como en su bibliografía y que no empleo datos de terceros sin la debida autorización, de acuerdo con la legislación vigente. Asimismo, declaro que soy plenamente consciente de que no respetar esta obligación podrá implicar la aplicación de sanciones académicas, sin perjuicio de otras actuaciones que pudieran iniciarse.

En Puerto Real, a 26 de junio de 2026

Fdo: Naím Rodríguez Reyes

Resumen

Este trabajo aborda el diseño de un sistema de navegación autónoma en interiores para un vehículo aéreo no tripulado (UAV), desarrollado bajo las restricciones de entornos industriales críticos donde es prioritario reducir el uso de redes inalámbricas para mitigar vulnerabilidades de seguridad. Para resolver este reto, se adaptó físicamente un dron comercial ligero DJI Tello mediante la impresión en 3D de un soporte rediseñado al que se le añade un espejo. Esta modificación desvía la visión de la cámara hacia el suelo con una inclinación calculada, proporcionando el campo visual cenital y anticipado necesario para seguir de manera precisa un circuito compuesto por una trayectoria continua y puntos de anclaje visual que ayudan a la estabilidad del vehículo.

El núcleo de la solución consiste en un software de visión por computador y control en tiempo real. Se utilizan técnicas de segmentación de imagen para estudiar la geometría de la pista, modulando dinámicamente la velocidad del dron según el tipo de tramo. La corrección de orientación y el centrado del dron sobre el eje del camino se gestionan de manera simultánea mediante una arquitectura de control en lazo cerrado.

La toma de decisiones estratégicas se resuelve a través de una lógica híbrida. El dron interpreta las señales de color emitidas por una tira de luces LED integrada en el circuito para escoger la ruta correcta en las bifurcaciones y ejecutar múltiples maniobras. Además, se incluye la detección activa de marcadores ArUco. Aunque este módulo sienta las bases para el desarrollo futuro de un mapa topológico, su utilidad actual se centra en la gestión de seguridad y contingencias en la planta, permitiendo interrumpir de forma autónoma el avance del dron ante imprevistos en la pista como averías o interferencias de maquinaria pesada.

Palabras clave: UAV, navegación autónoma, control de lazo cerrado, tiempo real, visión por computador, marcadores ArUco

Índice general

Índice de figuras	VI
-------------------	----

Índice de tablas	I
------------------	---

I Prolegómeno	1
----------------------	----------

1. Introducción	2
------------------------	----------

1.1. Contexto del proyecto	2
1.2. Motivación	2
1.3. Objetivos	3
1.4. Alcance y limitaciones	4
1.5. Estructura de la memoria	4
1.6. Metodología de desarrollo	5
1.7. Planificación	6
1.8. Cronograma	7

2. Estado del Arte	9
---------------------------	----------

2.1. Algoritmo de seguimiento de línea	9
2.1.1. Variantes en el algoritmo de extracción y seguimiento de trayectorias	9
2.1.2. Variantes en la naturaleza del objetivo visual	10
2.1.3. Estrategias de control en lazo cerrado (PID y PD)	10
2.2. Gestión de la velocidad de movimiento	11
2.3. Sistemas de toma de decisiones en navegación autónoma	12
2.3.1. Reconocimiento de patrones geométricos y marcadores estáticos	12
2.3.2. Segmentación cromática aplicada a la asignación dinámica de tareas	13

3. Contexto Teórico	14
----------------------------	-----------

3.1. Plataforma de vuelo	14
3.1.1. Especificaciones técnicas y limitaciones físicas	14
3.1.2. Sensores internos y estabilización de fábrica	15
3.1.3. Arquitectura de comunicación y control de software	15
3.2. Visión por computador y procesamiento de imágenes	16
3.2.1. Espacios de color: RGB frente a HSV	16
3.2.2. Técnicas de umbralización y segmentación	17
3.2.3. Morfología matemática y filtrado de ruido	17
3.3. Marcadores ArUco	19
3.4. Dinámica de vuelo y sistema de coordenadas del UAV	19
3.5. Sistema de control PID	21

3.5.1.	Análisis comparativo de arquitecturas de control (P, PI, PD y PID)	22
3.5.2.	Técnicas de sintonización y cálculo de coeficientes del controlador	23
3.5.3.	Aplicación a la cinemática del cuadricóptero	24
II	Desarrollo	25
4.	Hardware y Entorno Físico	26
4.1.	Modificación del sistema de visión del DJI Tello	26
4.1.1.	Diseño y evolución de la pieza 3D	26
4.1.2.	Especificaciones técnicas del conjunto óptico	27
4.2.	Diseño del entorno de pruebas	29
4.2.1.	Materiales y estructura de la pista	29
4.2.2.	Complejidad y restricciones geométricas del trazado	29
4.3.	Problemas de estabilidad y soluciones de hardware	31
4.3.1.	Fallo del Sistema de Posicionamiento por Visión (VPS)	31
4.3.2.	Implementación de puntos de anclaje visuales	31
5.	Sistema de Visión por Computador	32
5.1.	Adquisición y gestión de frames	32
5.1.1.	Control de flujo y gestión de nulos	32
5.1.2.	Algoritmo propio de comparación y descarte de duplicados	33
5.2.	Detección del estado luminoso	34
5.2.1.	Aislamiento en el espacio HSV	34
5.2.2.	Algoritmo de calibración probabilística por curvas gaussianas	35
5.2.3.	Mitigación de la asincronía temporal mediante filtrado de moda (Cola FIFO)	37
5.2.4.	Paleta cromática de comandos LED	38
5.3.	Preprocesamiento de imágenes	38
5.3.1.	Inversión de la imagen	39
5.3.2.	Reducción de escala	39
5.4.	Segmentación y binarización	39
5.4.1.	Transformaciones y suavizado de ruido	39
5.4.2.	Umbral adaptativo local	40
5.4.3.	Operaciones morfológicas y máscaras aisladas	40
5.5.	Análisis geométrico de la trayectoria	41
5.5.1.	Cálculo del índice de solidez	41
5.5.2.	Técnica de segmentación por ventanas de corte	42
5.5.3.	Selección de trayectoria	43
5.5.4.	Cálculo del desplazamiento lateral	44
5.6.	Modulación dinámica de la velocidad	44
5.7.	Integración de marcadores ArUco	45
6.	Sistema de Control y Navegación	46
6.1.	Máquina de estados y toma de decisiones	46
6.2.	Arquitectura controladores PID	47
6.2.1.	Evolución de la arquitectura del sistema	47

6.2.2. Sintonización de las constantes	47
6.3. Envío de comandos de vuelo	48
6.4. Modularidad y portabilidad del hardware	49
7. Pruebas y Resultados	50
7.1. Rendimiento del sistema de visión por computador	50
7.1.1. Telemetría y depuración gráfica en tiempo real	50
7.1.2. Robustez del sistema lumínico mediante calibración	51
7.1.3. Análisis temporal y de concurrencia	51
7.2. Evolución del sistema de guiado	53
7.2.1. Desarrollo del bucle de control	53
7.2.2. Justificación de técnica de sintonización del controlador	54
7.2.3. Perfil de velocidad dinámica	54
7.3. Análisis de la evolución global del proyecto	55
III Epílogo	56
8. Conclusiones y Trabajo Futuro	57
8.1. Lecciones aprendidas	57
8.2. Trabajo futuro	58
Información sobre Licencia	60
Bibliografía	61
IV Anexos	64
9. Manual de Instalación y Uso	65
9.1. Instalación y configuración del entorno	65
9.1.1. Requisitos del sistema	65
9.1.2. Despliegue del entorno virtual y dependencias	65
9.2. Manual de uso y operación	66
9.2.1. Parámetros y argumentos de configuración	66
9.2.2. Ejemplos de Ejecución	67
10. Manual Librería djitellopy	69
10.1. Manual de referencia de la API DJITelloPy y el SDK de Tello	69
10.1.1. Inicialización y enlace de comunicaciones	69
10.1.2. Gestión del flujo de vídeo	70
10.1.3. Comandos de acción física y altura	70
10.1.4. Guiado continuo	71

Índice de figuras

1.1. Ciclo de vida iterativo e incremental aplicado al desarrollo del sistema de guiado autónomo.	5
1.2. Diagrama gantt de la planificación del proyecto.	8
2.1. Proyectos de seguimiento de línea con drones DJI Tello y Parrot Mambo	9
2.2. Proyecto de seguimiento de mano con DJI Tello	10
2.3. Proyecto Line-flowing-DJI-TELLO-with-PD en funcionamiento	11
2.4. Proyecto DJITelloOpticalControl en funcionamiento	12
2.5. Proyecto Autonomous UAVs Swarm Mission en funcionamiento	13
3.1. Dron DJI Tello	14
3.2. Ejemplo de apertura y cierre de imagen binarizada con trazas	18
3.3. Marcador ArUco con ID = 0	19
3.4. Diagrama con los 3 ángulos de un dron.	20
3.5. Diagrama de un controlador PID	22
3.6. Comparación de respuestas de arquitecturas de control P, PI, PD y PID.	23
4.1. Render 3D del prototipo inicial en el software <i>Blender</i>	27
4.2. Comparativa de visiones con ambos prototipos	27
4.3. Dron DJI Tello con las modificaciones añadidas	28
4.4. Circuito de pruebas en un espacio cerrado	30
4.5. Plano del circuito de pruebas	30
5.1. Gráfico de barras comparativo entre el tiempo medio de ejecución del algoritmo propio y la función de NumPy (<code>np.array_equal</code>)	34
5.2. Representación numérica canal Hue (0-360)	36
5.3. Ejemplo de calibración de fronteras de curvas Gaussianas para los 6 colores	37
5.4. Pipeline de binarización adaptativa de un frame a color	41
6.1. Ordenador monoplaca Raspberry Pi Pico 2	49
7.1. Distribución de los valores Hue de cada color en una calibración	51
7.2. Diagrama Gantt del tiempo medio de ejecución de cada tarea del pipeline de visión.	52
7.3. Histograma de la distribución del tiempo de ejecución entre fotogramas.	52
8.1. Dron con ordenador monoplaca embebido en la parte superior.	59

Índice de tablas

5.1. Comparativa entre umbrales teóricos y después de la calibración.	37
7.1. Optimizaciones globales incorporadas al sistema.	55

Parte I

Prolegómeno

1. Introducción

La integración de la robótica y los vehículos aéreos no tripulados (*UAV* por sus siglas en inglés, Unmanned Aerial Vehicle) en entornos industriales ha supuesto un cambio de paradigma en la automatización de procesos, la logística y la monitorización de plantas. El uso de drones para la navegación en interiores presenta grandes ventajas frente a los vehículos terrestres no tripulados (*UGV* por sus siglas en inglés, Unmanned Ground Vehicle), principalmente por su capacidad para sortear obstáculos a nivel de suelo y moverse en tres dimensiones. No obstante, la navegación autónoma en interiores (donde no se dispone de señal GPS) requiere de sistemas avanzados de percepción espacial.

1.1. Contexto del proyecto

Este Trabajo de Fin de Grado nace a raíz de un proyecto propuesto a la empresa Navantia, enfocado a la mejora de operaciones en una de sus plantas industriales. En este tipo de entornos, la conectividad inalámbrica es un recurso crítico. Las infraestructuras de red en naves industriales a menudo se encuentran saturadas, son vulnerables a interferencias electromagnéticas propias de la maquinaria pesada, o simplemente están restringidas por protocolos de ciberseguridad. Depender de un gran ancho de banda por el medio inalámbrico para guiar un gran número de vehículos desde un servidor centralizado resulta inviable o altamente desaconsejable.

Ante este problema, surge la necesidad de desarrollar un sistema de navegación que delegue la toma de decisiones tanto al propio entorno, como a los sensores integrados del vehículo, minimizando así la necesidad de una comunicación bidireccional inalámbrica constante. La propuesta consiste en un dron capaz de seguir un trayecto físico definido en el suelo de la planta y reaccionar de manera autónoma a señales lumínicas localizadas. Mediante la captación y procesamiento de imágenes en tiempo real, el UAV interpreta un código de colores que dictaminará sus acciones de navegación inmediatas, operando de manera independiente a las interferencias de la red inalámbrica de la planta.

1.2. Motivación

La motivación principal de este proyecto es el desafío de ingeniería que supone adaptar una plataforma comercial de bajo coste y hardware cerrado, como es el dron DJI Tello, para cumplir con exigencias de carácter industrial. Resolver el problema de la dependencia de las redes inalámbricas requiere un enfoque centrado en la visión por computador y el procesamiento de vídeo en tiempo real.

Desde el punto de vista del diseño y la robótica, existe un reto físico considerable. El dron DJI Tello está equipado con una cámara frontal fija, diseñada para grabar el entorno hacia adelante. Para seguir una línea en el suelo, es imprescindible disponer de una visión cenital (hacia abajo). La imposibilidad de modificar el hardware del dron (sin realizar modificaciones destructivas) motiva la búsqueda de soluciones mediante diseño y fabricación de piezas 3D.

Asimismo, la motivación algorítmica reside en la creación de un flujo de procesamiento de imágenes altamente optimizado. Es necesario desarrollar funciones propias, capaces de superar en velocidad a librerías estándar en tareas específicas como la comparación de *frames*, y diseñar algoritmos de segmentación que aislen la ruta de posibles perturbaciones ambientales (sombras, reflejos o suciedad en el entorno industrial), todo esto en tiempo real.

1.3. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es diseñar, implementar y validar un sistema de navegación autónoma basado en visión por computador para un dron DJI Tello, permitiéndole seguir un trayecto físico complejo y tomar decisiones de ruta en tiempo real mediante el reconocimiento de señales lumínicas.

Para alcanzar este propósito general, se han definido los siguientes objetivos específicos:

- **Diseño e integración de hardware óptico:** Modelar en 3D e imprimir una pieza de acople que albergue un espejo para desviar el eje óptico de la cámara frontal hacia el suelo.
- **Desarrollo de un sistema de visión artificial en tiempo real:** Implementar un pipeline eficiente de preprocesado, binarización y extracción de características geométricas para identificar de manera inequívoca la línea de seguimiento sobre el circuito.
- **Implementación de reconocimiento de señales lumínicas:** Diseñar un protocolo de calibración y detección para interpretar los comandos emitidos por una tira LED (parada, giros, retroceso, aterrizaje).
- **Desarrollo del sistema de control de vuelo:** Diseñar e integrar controladores PID para regular el ángulo de guiñada (yaw) y el desplazamiento lateral (roll), garantizando que el dron se mantenga sobre la trayectoria deseada calculando el error geométrico en cada frame.
- **Integración de marcadores ArUco:** Integración de la detección de marcadores para su uso como advertencia temporal ante averías en la planta.

1.4. Alcance y limitaciones

El alcance de este proyecto abarca desde la adaptación del hardware en el vehículo hasta el desarrollo del software de control y la validación física en un circuito de pruebas. El entorno de validación consiste en un circuito de tela blanca con un trazado de cinta adhesiva negra que actúa como guía, complementado con puntos de anclaje (pegatinas) para asistir a la estabilización por flujo óptico del dron, y un sistema LED integrado para la señalización.

Las limitaciones del trabajo vienen marcadas de manera estricta por las características técnicas de la plataforma DJI Tello y por las restricciones asumidas en el diseño del circuito:

- **Limitaciones de la plataforma aérea:** El dron tiene un peso total de 80 gramos (batería incluida) y una autonomía máxima de vuelo aproximada de 13 minutos. Esto restringe severamente la carga útil, limitando el acople del espejo y su soporte a un peso y a unas dimensiones concretas para evitar desequilibrar el centro de gravedad e imposibilitar el vuelo.
- **Procesamiento de video:** La captura de vídeo está limitada a una resolución de 960x720 píxeles a 30 fotogramas por segundo (fps) con un campo de visión de 82.6 grados. Aunque el objetivo a nivel industrial es evitar la dependencia del Wi-Fi, debido a que el hardware del Tello es cerrado y no posee capacidad de cómputo interno accesible para el usuario, ni posee la capacidad de conexión vía alámbrica, el procesamiento se realiza forzosamente enviando los frames a un ordenador externo vía Wi-Fi para esta prueba de concepto.
- **Restricciones geométricas del trayecto:** El circuito presenta restricciones de diseño específicas para garantizar la estabilidad algorítmica: todos los radios de las curvas son de 90 centímetros y las intersecciones o bifurcaciones están limitadas a un máximo de dos salidas o entradas.

1.5. Estructura de la memoria

Para detallar el desarrollo y los resultados del proyecto, esta memoria se organiza en los siguientes capítulos:

- **Capítulo 2:** Estado del arte. Revisión superficial del estado actual de proyectos con características similares, así como las carencias y soluciones que se proponen ante los problemas sin resolver de estos.
- **Capítulo 3:** Marco teórico. Se revisan los fundamentos de la visión por computador, las características del DJI Tello y la teoría subyacente a los controladores PID y los marcadores ArUco.
- **Capítulo 4:** Hardware y entorno físico. Detalla las modificaciones realizadas sobre el dron, incluyendo el diseño 3D del soporte del espejo y la justificación

de su ángulo, así como la construcción del circuito de pruebas y las soluciones aportadas a los problemas de estabilidad.

- **Capítulo 5:** Sistema de visión por computador. Expone de manera extensa la primera etapa del bucle lógico del programa: optimización en la recepción de frames, calibración del espacio de colores, binarización, cálculo geométrico y selección de figuras de seguimiento.
- **Capítulo 6:** Sistema de control y navegación. Describe la toma de decisiones basada en las entradas visuales, el cálculo de trayectorias, y la configuración técnica y ajuste de los controladores PID encargados de enviar los comandos de vuelo al dron.
- **Capítulo 7:** Pruebas y resultados. Analiza los tiempos de cómputo de las diferentes tareas del pipeline de procesamiento y la robustez del sistema de calibración LED, así como un análisis de la evolución general del proyecto.
- **Capítulo 8:** Conclusiones y trabajo futuro. Sintetiza los logros del proyecto e introduce posibles mejoras para acercar esta prueba de concepto a su implantación final en un entorno industrial real.

1.6. Metodología de desarrollo

Para aportar una visión estructurada del flujo de ingeniería aplicado, la Figura 1.1 ilustra el carácter cíclico de este enfoque. Cada iteración completada no solo ha supuesto un incremento en las líneas de código del software, sino una evolución directa en la complejidad del circuito físico y en las capacidades de navegación del UAV.

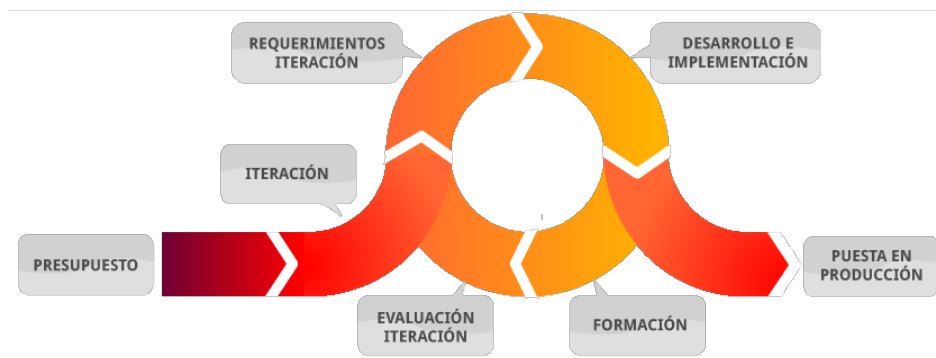


Figura 1.1: Ciclo de vida iterativo e incremental aplicado al desarrollo del sistema de guiado autónomo.

A continuación, se detallan las tareas ejecutadas dentro de cada una de las fases que conforman las iteraciones del proyecto:

- **Requerimientos:** Esta fase inicial de la iteración se centra en la identificación de necesidades funcionales, la búsqueda de soluciones teóricas y el análisis exhaustivo de las restricciones físicas y computacionales. Aquí se estudió la documentación del SDK de Ryze Robotics, la abstracción hardware de *djitellopy* y las funciones

de binarización de *OpenCV*. En este bloque se definieron los límites críticos del sistema, como el descarte de algoritmos de visión pesados que comprometieran la tasa de 30 frames por segundo de la cámara frontal, o la determinación de las dimensiones geométricas de la pista necesarias para la correcta navegación del dron.

- **Desarrollo:** Una vez consolidados los requerimientos y viabilidad de la iteración, se procede a la traducción de los conceptos lógicos en código Python. En esta fase se programaron las clases de control de los reguladores *PID* independientes (tanto para el lazo de guiñada o *Yaw* como para el de alabeo o *Roll*), el aislamiento de color en el espacio HSV dentro de *vision.py* para el tratamiento de la tira LED y la lógica de prioridades para los marcadores ArUco.
- **Formación:** En el contexto de la robótica y los sistemas autónomos, esta fase alude a la configuración y evolución del entorno operativo sobre el cual se despliega el sistema. Equivale a la construcción y mejora progresiva del circuito físico de pruebas. En este bloque se ejecutó el diseño y unión de las telas para formar rectas e intersecciones, la instalación de la tira de diodos LED sobre el eje del camino, la colocación de pegatinas negras como puntos de anclaje visual en los márgenes y la distribución estratégica de las balizas ArUco para preparar el entorno de vuelo.
- **Evaluación:** El ciclo cierra con la fase de verificación experimental del prototipo sobre el circuito. Utilizando el DJI Tello real, se ejecutaron pruebas de vuelo en lazo cerrado para monitorizar el comportamiento de la aeronave, comprobar la robustez del sistema de visión ante cambios de luz ambiental y medir los tiempos de ejecución de procesamiento. Si la evaluación detectaba inestabilidades, como desestabilizaciones en las curvas o derivas por acumulación de error, los resultados realimentaban de inmediato el inicio de una nueva iteración para reajustar las ganancias del PID o refinar el filtrado de imágenes.

Este flujo de trabajo iterativo ha permitido mitigar riesgos de manera temprana, adaptando el software a las limitaciones cinemáticas reales del dron a medida que aumentaba la complejidad del circuito.

1.7. Planificación

El desarrollo global de este Trabajo Fin de Grado se ha extendido a lo largo de aproximadamente nueve meses, comprendidos entre septiembre de 2025 y junio de 2026. El proyecto se ha dividido estratégicamente en fases mensuales, distribuyendo la carga de trabajo desde la familiarización inicial con el hardware hasta la validación de los sistemas de navegación más complejos.

A continuación, se desglosa una planificación teórica de las tareas a un nivel alto de abstracción:

1. **Familiarización inicial y optimización del acople óptico:** Estudio del SDK nativo de Ryze Robotics, familiarización con la biblioteca *djitellopy* y rediseño

en software de modelado 3D de la pieza de soporte del espejo para optimizar el campo de visión de la cámara hacia el suelo.

2. **Prototipado del circuito base y procesamiento de imagen primario:** Confección de la pista inicial en línea recta simplificada y desarrollo de los primeros módulos de visión artificial dedicados al filtrado, umbralizado y binarización geométrica de la trayectoria de cinta adhesiva.
3. **Diseño e implementación del control en lazo cerrado:** Modelado matemático, programación y sintonización de los controladores PID discretos dedicados a corregir de forma autónoma la orientación (*Lazo de Yaw*) y la deriva lateral del dron (*Lazo de Roll*).
4. **Integración multisensorial y calibración cromática:** Despliegue de la tira de diodos LED sobre el circuito, desarrollo del algoritmo automatizado de calibración previa en el espacio HSV frente a cambios de luz ambiental, e integración de marcadores ArUco como balizas de decisión.
5. **Navegación avanzada y validación integrada del sistema:** Confección de un circuito de mayor complejidad con bifurcaciones, programación de la lógica de prioridades para la toma de decisiones en vuelo, implementación del algoritmo de gestión dinámica de la velocidad lineal y validación del sistema completo ante múltiples situaciones.
6. **Análisis de métricas y documentación:** Monitorización y recolección estadística de los tiempos de cómputo de los procedimientos de visión artificial, análisis del comportamiento cinemático y redacción de la memoria del proyecto.

1.8. Cronograma

Para ofrecer una visión clara del solapamiento de las tareas, la Figura 1.2 resume la distribución temporal estimada del proyecto para cada tarea diferenciadas en 4 campos distintos: *Hardware*, *Software*, *Ensayos* y *Documentación*.

La planificación temporal de este proyecto no sigue un modelo en cascada tradicional, sino que se ha estructurado bajo un enfoque iterativo e incremental, donde el desarrollo del software y las modificaciones del hardware se retroalimentan constantemente de los ensayos. Por este motivo, la tarea de *Ensayos y extracción de métricas* se plantea como un eje transversal que abarca prácticamente la totalidad del proyecto. Al tratarse de un sistema dinámico como un dron, la viabilidad de cada algoritmo de visión o la sintonización de los lazos de control no puede validarse únicamente mediante simulación, sino que requiere pruebas inmediatas en el circuito de pruebas para evaluar el comportamiento real del cuadricóptero ante perturbaciones.

Esta naturaleza cíclica justifica también retomar subtareas en meses posteriores debido a esta metodología. Por ejemplo, el bloque de *Procesado de imagen* se retoma en el mes de enero debido a la dependencia con la *Ampliación geométrica del circuito*, ya que la transición de una pista rectilínea simple a un entorno complejo con curvas e intersecciones exige refinar los algoritmos relativos a la binarización. Finalmente, la

Redacción memoria y vídeos demostrativos se ha posicionado en el último tercio del proyecto, debido al riesgo de introducir modificaciones estructurales en los controladores dinámicos durante las fases críticas de ensayo, reflejando fielmente el diseño final de software y hardware del proyecto.

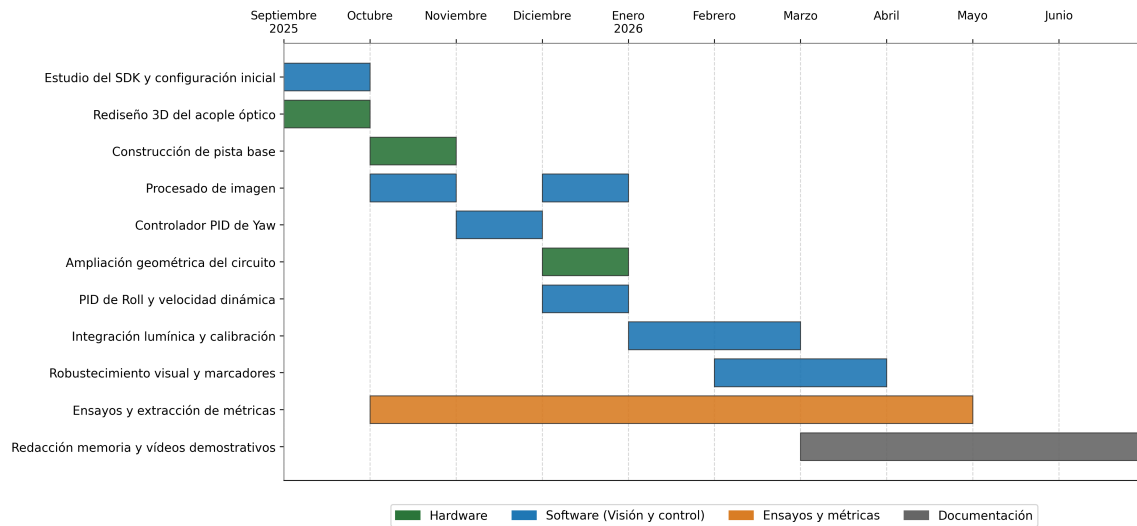


Figura 1.2: Diagrama gantt de la planificación del proyecto.

2. Estado del Arte

2.1. Algoritmo de seguimiento de línea

La navegación autónoma mediante el seguimiento de trayectorias es un enfoque ampliamente estudiado en la robótica aérea. En el estado actual es frecuente encontrar proyectos que emplean drones comerciales ligeros, como el modelo DJI Tello [1] [2], para desarrollar sistemas de navegación visual en interiores utilizando herramientas de visión por computador similares a las de este trabajo (Python y OpenCV).

También se incluyen investigaciones donde se alcanza el mismo objetivo de guiado mediante el uso de arquitecturas de hardware diferentes y frameworks de software alternativos [3] [4].



Figura 2.1: Proyectos de seguimiento de línea con drones DJI Tello y Parrot Mambo

2.1.1. Variantes en el algoritmo de extracción y seguimiento de trayectorias

Dentro del estado del arte enfocado exclusivamente al seguimiento de líneas en el suelo, existen alternativas matemáticas al cálculo tradicional de centroides de contornos binarizados.

Proyectos enmarcados en entornos competitivos de navegación autónoma aérea [5], proponen el uso del algoritmo de trazado de líneas de Bresenham. Esta aproximación altera la estrategia de visión artificial: en lugar de analizar el plano de la imagen de forma global, se trazan vectores de escaneo para localizar la transición de píxeles negros. Aunque parece una solución ingeniosa, carece de una flexibilidad para

geometrías como las curvas, dificultando un trazado fiable de las mismas, así como las bifurcaciones o intersecciones.

2.1.2. Variantes en la naturaleza del objetivo visual

Es relevante destacar que el uso del DJI Tello como plataforma móvil visual no se limita únicamente al seguimiento de líneas continuas. El estado del arte demuestra una fuerte línea de investigación orientada al rastreo y reconocimiento de características biométricas dinámicas.

Desarrollos enfocados en el procesamiento de rostros en tiempo real, tales como *DJI-TELLO-Face-Tracking* [6] o *Tello-Face-Recognition* [7], sustituyen la detección del camino en el suelo por la extracción de cajas delimitadoras y descriptores faciales mediante redes neuronales ligeras. Si bien el objetivo es diferente, estos proyectos validan el uso del DJI Tello como una plataforma robusta para la ejecución de bucles de control basados en el error de píxeles respecto al centro óptico de la cámara frontal para el seguimiento de diferentes objetivos.

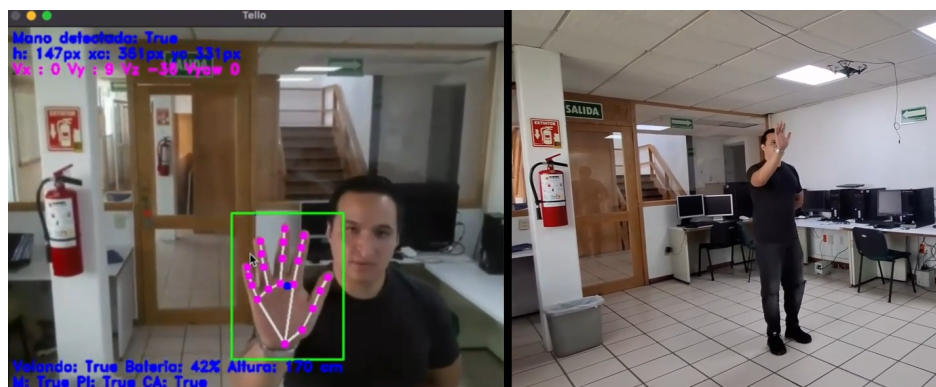


Figura 2.2: Proyecto de seguimiento de mano con DJI Tello. ¹

2.1.3. Estrategias de control en lazo cerrado (PID y PD)

Independientemente de la metodología de extracción visual elegida, la estabilidad cinemática del UAV en el espacio tridimensional fundamenta su éxito en sistemas de control realimentados. La integración de controladores de lazo cerrado destaca como un estándar indiscutible en la robótica aérea contemporánea [8].

Aunque teóricamente el controlador PID completo (Proporcional-Integral-Derivativo) es la opción más extendida, en circuitos cerrados sobre drones es frecuente omitir la acción Integral (I) (puesto que las derivas externas son inexistentes), derivando en estructuras simplificadas Proporcional-Derivativas (PD). Un claro ejemplo de esta tendencia se encuentra en el repositorio público *Line-flowing-DJI-TELLO-with-PD* [9], el cual implementa una arquitectura de control parecida a la planteada en este trabajo:

¹Vídeo del funcionamiento - <https://www.youtube.com/watch?v=8IIsz5iFlA4>

un lazo cerrado PD gobernando un dron DJI Tello para seguir líneas en el suelo. Sin embargo, el análisis de dicho proyecto pone de manifiesto una sintonización deficiente de las ganancias K_p y K_d . En sus resultados documentados, se observa cómo el vehículo presenta una respuesta lenta y un comportamiento subamortiguado, crítico al aproximarse a las transiciones curvas, provocando balanceos inestables.

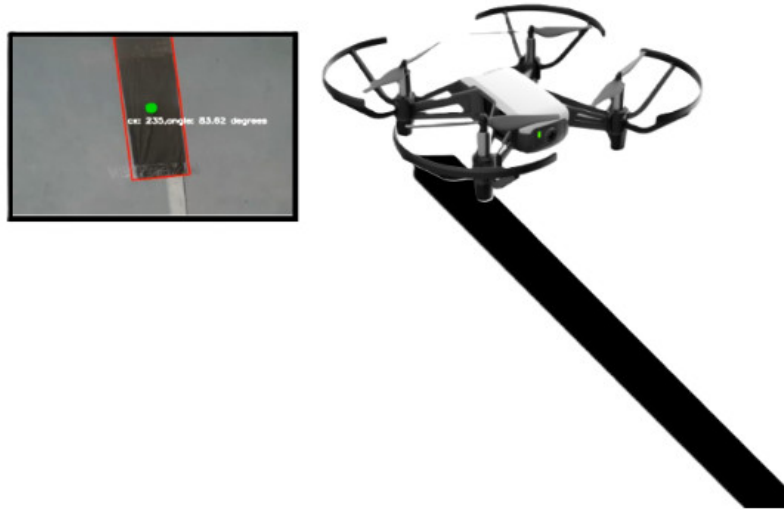


Figura 2.3: Proyecto *Line-flowing-DJI-TELLO-with-PD* en funcionamiento ²

2.2. Gestión de la velocidad de movimiento

Uno de los factores limitantes en las soluciones actuales de seguimiento de línea es la gestión de la velocidad lineal del vehículo. La mayoría de los proyectos optan por configurar una velocidad constante durante todo el recorrido. Estas velocidades suelen restringirse a rangos pequeños, habitualmente por debajo de los 10 cm/s [1] o fijas en los 15 cm/s [2].

Mantener una velocidad fija penaliza la eficiencia en los tramos rectos, donde se podría acelerar de forma segura sin comprometer su estabilidad. En capítulos posteriores se expondrá la velocidad dinámica según varias características geométricas del circuito y posicionales del dron como solución a este problema.

²Vídeo de prueba Line-flowing-DJI-TELLO-with-PD - <https://www.youtube.com/watch?v=CcdG9qiCEZA>

2.3. Sistemas de toma de decisiones en navegación autónoma

Para superar las limitaciones del guiado pasivo y dotar a un dron de la capacidad de tomar decisiones lógicas en entornos complejos (como parar, aterrizar o seleccionar un camino), el estado del arte recurre habitualmente a la detección de estímulos visuales artificiales en el entorno.

2.3.1. Reconocimiento de patrones geométricos y marcadores estáticos

Una de las aproximaciones tradicionales para la toma de decisiones se basa en el reconocimiento de patrones geométricos. Un ejemplo de esta metodología se encuentra en el proyecto *Tello Drone Arrow Navigator* [10], en el cual el vehículo aéreo toma decisiones direccionales al identificar formas de flechas en el suelo mediante el análisis de contornos y envolturas convexas (*convex hull*). No obstante, este enfoque presenta la desventaja de que los elementos de señalización son estáticos, lo cual imposibilitaría un cambio temporal o poder cambiar de decisión de manera dinámica.

Como alternativa para asociar de manera única un comando de control a una marca visual, se han utilizado marcadores ArUco. El desarrollo *DJITelloOpticalControl* [11] ejemplifica esto mediante el uso exclusivo de marcadores ArUco colocados en la escena. En dicha arquitectura, el sistema de visión decodifica el marcador obteniendo la posición tridimensional y el ID. Utiliza el identificador único para recorrerlos de forma ascendente (comenzando por el 0) y colocándose frente a cada uno de cierta manera antes de continuar al siguiente identificador.



Figura 2.4: Proyecto *DJITelloOpticalControl* en funcionamiento ³

³Vídeo de prueba *DJITelloOpticalControl* - <https://www.youtube.com/watch?v=B8aU0DVYYco>

2.3.2. Segmentación cromática aplicada a la asignación dinámica de tareas

El estado del arte también contempla el uso de la segmentación cromática pura como un mecanismo eficiente para la asignación de comandos en sistemas autónomos. Esta aproximación destaca especialmente en escenarios que requieren una reacción inmediata ante estímulos visuales cambiantes.

Un ejemplo de esto es el proyecto de navegación cooperativa *Autonomous UAVs Swarm Mission* [12]. En este desarrollo, los drones procesan el entorno visual en tiempo real para identificar objetivos cromáticos tridimensionales (globos de distintos colores). El sistema de control asocia cada objetivo a una aeronave concreta mediante filtrado en el espacio HSV, utilizando el color no solo para el guiado físico, sino como la orden directa para iniciar la maniobra de interceptación (destrucción del globo).

Este enfoque reactivo se alinea estrechamente con la propuesta de este trabajo. Demuestra que utilizar el color y las señales lumínicas es una alternativa muy ligera a nivel computacional (como veremos en el apartado de resultados (7.1.3)), ideal para que drones con recursos limitados tomen decisiones en tiempo real sin saturar el procesador.



Figura 2.5: Proyecto *Autonomous UAVs Swarm Mission* en funcionamiento ⁴

⁴Vídeo de prueba *Autonomous UAVs Swarm Mission* - <https://www.youtube.com/watch?v=y370Tdfmgqc>

3. Contexto Teórico

Este capítulo aborda las características técnicas de la plataforma de vuelo seleccionada, evaluando tanto sus capacidades de fábrica como sus interfaces de programación. En segundo lugar, se exponen los fundamentos teóricos del procesamiento de imágenes y la visión por computador, detallando las técnicas de segmentación, operaciones morfológicas y análisis geométrico necesarias para la interpretación de la pista. También se introduce el concepto de marcadores fiduciales ArUco. Finalmente, se examina la teoría de los sistemas de control en lazo cerrado a través de los controladores Proporcional-Integral-Derivativo (PID).

3.1. Plataforma de vuelo

La elección del vehículo aéreo es un factor condicionante para el diseño de cualquier algoritmo de navegación en interiores. En este proyecto se ha seleccionado el cuadricóptero DJI Tello, un dispositivo comercial de bajo coste desarrollado por Ryze Robotics. Este dispositivo está diseñado principalmente para entornos educativos, lo que implica una serie de ventajas pero también varias restricciones de hardware que el software de control debe ser capaz de manejar.



Figura 3.1: Dron DJI Tello ¹

3.1.1. Especificaciones técnicas y limitaciones físicas

El DJI Tello destaca por su ligereza, con un peso de apenas 87 gramos con la batería integrada y los protectores de hélices. Esta característica lo convierte en un dispositivo seguro para vuelos en espacios cerrados, minimizando los riesgos de daños materiales o personales en caso de colisión. Sin embargo, este reducido peso limita su capacidad de carga útil, lo que restringe la adición de sensores o elementos mecánicos

¹DJI Store Iberia - <https://www.djistoreiberia.com/dji-tello/>

externos a unos pocos gramos para no desestabilizar el centro de masa ni sobrecargar los motores sin escobillas.

La autonomía de vuelo se sitúa en torno a los 13 minutos bajo condiciones ideales, un tiempo que se ve reducido en la práctica debido al consumo del procesador interno y la transmisión constante de vídeo. El vehículo cuenta con un campo de visión (*FOV*) de 82.6° a través de una cámara frontal fija, la cual captura secuencias de vídeo con una resolución nativa de 960x720 píxeles a una tasa de refresco de 30 fotogramas por segundo (*fps*), así como captura de imágenes con una resolución de 2592x1936 píxeles. Cuenta además con estabilización de imagen electrónica, permitiendo una mayor nitidez de vídeo incluso con el dron en movimiento.

3.1.2. Sensores internos y estabilización de fábrica

A pesar de su reducido tamaño, el dron incorpora una unidad de procesamiento centralizada encargada de la estabilidad del vuelo a través de una Unidad de Medición Inercial (*IMU*) encargada de medir la velocidad, orientación y fuerzas gravitacionales. Para paliar la ausencia de señal GPS en interiores, el Tello implementa un Sistema de Posicionamiento de Visión [13] (*VPS*, Vision Positioning System). Este sistema, ubicado en la parte inferior del chasis, combina una cámara de baja resolución orientada al suelo y un sensor de ondas infrarrojas 3D. El VPS calcula el flujo óptico del terreno para estimar el desplazamiento lateral y longitudinal, permitiendo que el dron realice un vuelo estacionario relativamente estable. Cabe resaltar que este sistema depende críticamente de la existencia de texturas o contrastes en el suelo; superficies completamente homogéneas o monocromáticas, así como superficies con patrones o texturas idénticas anulan la efectividad del VPS, induciendo pequeños movimientos indeseados.

3.1.3. Arquitectura de comunicación y control de software

El DJI Tello carece de capacidad de cómputo a bordo accesible para el usuario para ejecutar algoritmos pesados de visión artificial. Por ello, la arquitectura del sistema sigue un modelo cliente-servidor. El dron actúa como un punto de acceso Wi-Fi (estándar 802.11n a 2.4 GHz) al que se conecta el servidor (en este caso, un ordenador personal).

La comunicación se gestiona mediante el protocolo de transporte UDP [14] (*User Datagram Protocol*), escogido por su baja latencia en la transmisión de datos en tiempo real, asumiendo el riesgo de pérdida ocasional de paquetes. El intercambio de información se realiza a través de tres puertos principales:

- **Puerto de comandos (8889):** Permite enviar cadenas de texto en formato ASCII (comandos SDK) para ordenar despegues, aterrizajes o movimientos cinemáticos, y recibir confirmaciones (`ok` o `error`).
- **Puerto de estado (8890):** El dron retransmite de forma pasiva y periódica un string con datos telegráficos como el nivel de batería, la altura, la velocidad en los tres ejes y la aceleración.

- **Puerto de vídeo (11111):** El dron codifica el flujo de la cámara en formato H.264 y lo envía mediante paquetes UDP a la estación de trabajo.

Para interactuar con esta arquitectura desde un entorno de desarrollo moderno, se ha utilizado el lenguaje de programación Python (versión 3.12.3) junto con la librería especializada *djitellopy* (versión 2.4.0). Esta librería proporciona una API de alto nivel en programación orientada a objetos, gestionando de forma automática los hilos de ejecución necesarios para la recepción del stream de vídeo, la lectura de la telemetría y el envío de comandos de velocidad sin bloquear el bucle principal del algoritmo de control (aunque existen varios comandos bloqueantes que impiden la recepción de stream de vídeo en tiempo real) y la lógica del programa.

3.2. Visión por computador y procesamiento de imágenes

El núcleo perceptivo del proyecto radica en la interpretación de los cambios cromáticos y geométricos del circuito a partir de la secuencia de vídeo capturada. Para lograr un procesamiento robusto en tiempo real, es necesario recurrir a herramientas estándar de la visión artificial, como lo es la librería *OpenCV*² (versión 4.10.0.84).

3.2.1. Espacios de color: RGB frente a HSV

Las imágenes digitales se capturan originalmente en el espacio de color RGB (*Red*, *Green*, *Blue*), donde cada píxel se define por la combinación de intensidades de estos tres canales primarios. Aunque el espacio RGB es óptimo para la visualización en pantallas, resulta sumamente ineficiente para tareas de segmentación cromática en entornos reales. La razón es que la información del color (crominancia) y la información de la iluminación (luminancia) se encuentran entremezcladas en los tres canales. Cualquier cambio en la luz ambiental (sombras o reflejos) altera por completo los valores de R, G y B, provocando fallos en los umbrales de detección previamente definidos.

Para solventar esta limitación, el flujo de trabajo requiere la conversión de la imagen al espacio de color HSV (*Hue*, *Saturation*, *Value*, o Matiz, Saturación y Valor). En este modelo:

- **Matiz** (H - Hue): Representa el color puro o la longitud de onda de la luz (por ejemplo, rojo, verde o azul) mapeado en una circunferencia de 0 a 360 grados (codificado en OpenCV de 0 a 179 para mayor eficiencia de memoria empleando enteros de 8 bits). Este canal es en gran medida independiente de los cambios de iluminación.
- **Saturación** (S - Saturation): Indica la pureza o viveza del color, variando desde el gris (0 %) hasta el color totalmente saturado (100 %).

²OpenCV - <https://opencv.org/>

- **Valor** (V - Value): Determina la intensidad de la luz, simulando el brillo percibido.

Al aislar el canal H , se pueden definir regiones específicas para identificar el color de los diodos LED con una alta tolerancia a los reflejos exteriores o a la pérdida de brillo provocada por la distancia.

3.2.2. Técnicas de umbralización y segmentación

La umbralización es el proceso de conversión de una imagen en escala de grises a una imagen binarizada, donde los píxeles adquieren un valor binario: un valor alto (255, blanco) para el objeto de interés y un valor bajo (0, negro) para el fondo.

En entornos industriales no controlados, el uso de un umbral global fijo resulta ineficaz debido a que la iluminación no es homogénea a lo largo de toda la pista. Por ello, se recurre a la umbralización adaptativa. A diferencia del método global fijo, la umbralización adaptativa calcula un umbral específico para cada píxel individual de la imagen en función de las características de su vecindad ($V(x, y)$). El algoritmo examina un bloque cuadrado de píxeles de tamaño impar ($N \times N$) centrado en el píxel objetivo. En la variante de umbralización adaptativa por media (*Adaptive Mean Thresholding*) [15], el valor del umbral $T(x, y)$ se calcula como la media aritmética de los píxeles vecinos menos una constante de ajuste C :

$$T(x, y) = \frac{1}{N^2} \sum_{(i,j) \in V(x,y)} I(i, j) - C \quad (3.1)$$

$$AdaptiveThreshold(x, y) = \begin{cases} 255 & \text{si } I(x, y) < T(x, y) \\ 0 & \text{si } I(x, y) \geq T(x, y) \end{cases} \quad (3.2)$$

Si el valor del píxel analizado es menor que el umbral calculado $T(x, y)$, se asigna al fondo; de lo contrario, se asigna al objeto. Este enfoque adaptativo permite aislar la cinta adhesiva negra de la tela blanca con una alta fiabilidad frente a gradientes de luz superficiales, sombras proyectadas por el propio cuerpo del dron o destellos de luz sobre el material plástico de la cinta adhesiva.

3.2.3. Morfología matemática y filtrado de ruido

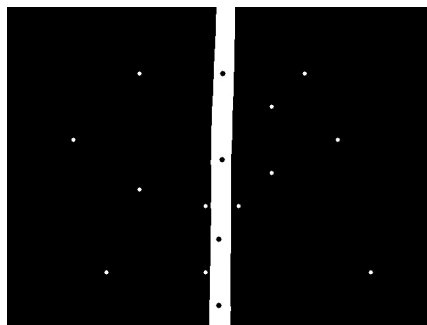
La binarización de una imagen suele contener imperfecciones en forma de pequeños puntos blancos aislados o discontinuidades en los bordes de la figura principal debido a irregularidades físicas del terreno, manchas o sombras. Para subsanar estos defectos sin alterar la geometría general de los objetos, se suelen aplicar técnicas de morfología matemática, las cuales consisten en la convolución de la imagen binaria con una matriz pequeña denominada kernel.

Las dos operaciones morfológicas fundamentales son la erosión y la dilatación:

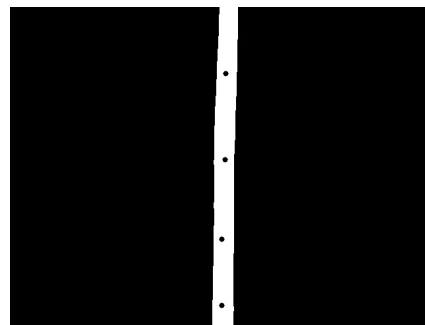
- **Erosión:** Reduce los límites del objeto binario. Un píxel de la imagen original se mantiene en blanco solo si todos los píxeles cubiertos por el kernel son blancos. Esto elimina elementos brillantes aislados que sean más pequeños que el tamaño del elemento estructurante.
- **Dilatación:** Expande los límites del objeto. Un píxel se transforma en blanco si al menos un píxel de la vecindad cubierta por el kernel es blanco. Su utilidad radica en la unión de partes fragmentadas de una misma línea.

A partir de estas dos operaciones básicas se construyen operaciones compuestas cruciales para el proyecto:

- **Apertura** (*Opening*): Consiste en la aplicación sucesiva de una erosión seguida de una dilatación. Su propiedad principal es eliminar pequeños objetos o filamentos no deseados (como pelusas o manchas finas en la tela blanca) manteniendo la forma original de los objetos grandes.
- **Cierre** (*Closing*): Consiste en una dilatación seguida de una erosión. Se utiliza para rellenar pequeños huecos, fisuras o grietas internas presentes dentro del objeto binario (pequeños destellos de luz en la cinta adhesiva negra), asegurando la continuidad de la línea de guiado.



(a) Imagen binarizada



(b) Imagen binarizada tras apertura



(c) Imagen binarizada tras cierre



(d) Imagen binarizada tras apertura y cierre

Figura 3.2: Ejemplo de apertura y cierre de imagen binarizada con trazas

3.3. Marcadores ArUco

Para dotar al sistema de navegación de capacidades de localización espacial absoluta y reconocimiento de hitos semánticos sin incurrir en costes computacionales excesivos, se introduce el uso de marcadores artificiales. En el ámbito de la robótica y la visión por computador, el sistema ArUco [16] destaca como uno de los estándares más utilizados.

Un marcador ArUco es un patrón cuadrado bidimensional compuesto por un borde exterior negro continuo y una matriz binaria interior (celdas blancas y negras) que codifica un identificador numérico único. El borde exterior negro facilita la detección rápida del contorno de la figura en la imagen binarizada, permitiendo localizar los cuatro vértices del marcador incluso bajo condiciones de deformaciones de perspectiva. La matriz interna se evalúa mediante códigos de corrección de errores, lo que garantiza una tasa de falsos positivos extremadamente baja y una alta robustez frente a oclusiones parciales.

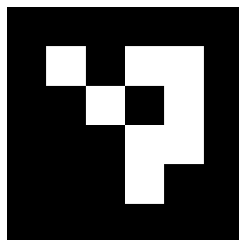


Figura 3.3: Marcador ArUco con ID = 0

En el contexto de este proyecto, los marcadores ArUco se introducen con la finalidad a largo plazo de diseñar un mapa topológico para la navegación del entorno industrial. Sin embargo, su utilidad actual se centra en la resolución de contingencias en la planta. Esto permite establecer puntos de control ante imprevistos, donde la detección de un identificador específico indica al dron la necesidad de cortar o interrumpir el camino actual debido a la presencia temporal de maquinaria o a una avería técnica en ese tramo de la pista.

3.4. Dinámica de vuelo y sistema de coordenadas del UAV

Para comprender el comportamiento cinemático del DJI Tello y el diseño de los lazos cerrados de control, es imprescindible definir el sistema de referencia físico de la aeronave. A diferencia de un vehículo terrestre ordinario, un cuadricóptero opera en un espacio tridimensional donde su actitud y orientación espacial se definen mediante la combinación de movimientos y rotaciones en torno a sus tres ejes principales: alabeo (*roll*), cabeceo (*pitch*) y guiñada (*yaw*).

Estas rotaciones, que determinan los ángulos de navegación o de actitud de la aeronave, describen la orientación del cuerpo móvil respecto a un sistema de coordenadas de referencia local fijo al dron, cuyo origen se sitúa de manera exacta en su centro de masas:

- **Alabeo (Roll, ϕ):** Representa la rotación de la aeronave en torno a su eje longitudinal (eje X en el plano horizontal, apuntando hacia el frente del dron). Un ángulo de alabeo distinto de cero inclina lateralmente el plano de los motores, lo que genera un empuje que desplaza al dron horizontalmente hacia la izquierda o hacia la derecha. En este proyecto, este eje es gobernado de forma autónoma por el Lazo de Alabeo para corregir la desviación lateral respecto al centroide de la línea de cinta adhesiva.
- **Cabeceo (Pitch, ζ):** Define la rotación en torno al eje transversal (eje Y en el plano horizontal, perpendicular al avance). Modificar el cabeceo provoca que el morro del dron se incline hacia abajo (avance longitudinal hacia adelante) o hacia arriba (retroceso). Dentro de la arquitectura desarrollada, el cabeceo no se regula mediante un bucle cerrado de error geométrico, sino que se parametriza dinámicamente como una velocidad de avance constante o modulada en función de la solidez y linealidad del camino detectado por el sistema de visión.
- **Guiñada (Yaw, ψ):** Expresa la rotación en torno al eje vertical (eje Z, perpendicular al plano de los motores y apuntando hacia el suelo). Cambiar la guiñada modifica la orientación del dron, haciéndolo rotar sobre su propio centro de gravedad. Esta acción altera la dirección hacia la que apunta la cámara frontal. En el sistema implementado, el Lazo de Guiñada monitoriza activamente este ángulo para asegurar que el vector de avance del UAV permanezca paralelo a la dirección que traza la pista en los tramos curvos y rectos.

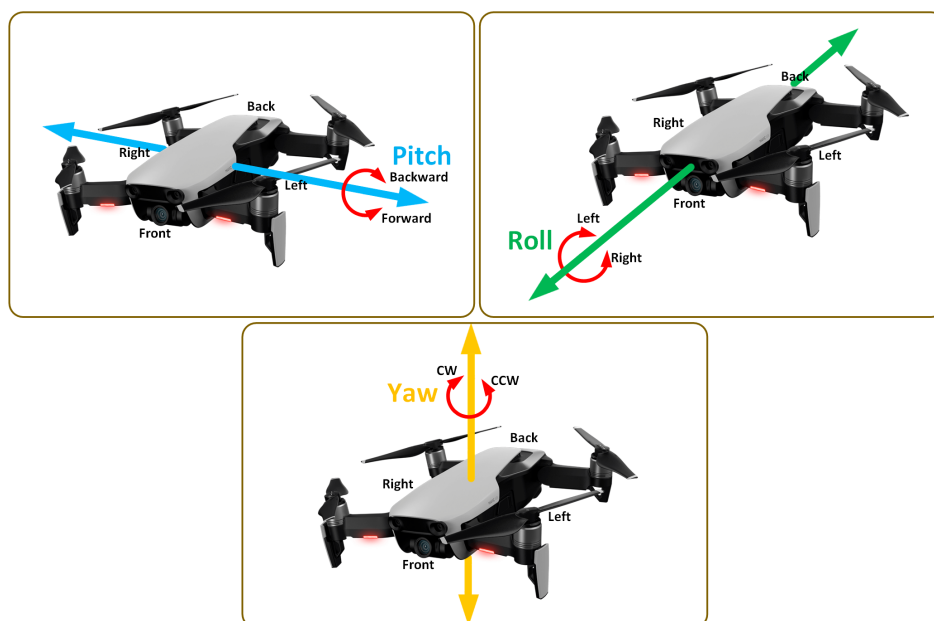


Figura 3.4: Diagrama con los 3 ángulos de un dron. ³

³Modificación de la imagen original <https://ar5iv.labs.arxiv.org/html/2208.12328>

3.5. Sistema de control PID

Una vez que el sistema de visión ha procesado el frame y determinado la desviación geométrica del dron respecto a la trayectoria ideal, es necesario traducir este error en acciones físicas coordinadas sobre los motores. El mecanismo estándar de la ingeniería de control para resolver este problema en lazo cerrado es el Controlador Proporcional-Integral-Derivativo (*PID*).

El propósito de un controlador PID es minimizar de forma continua el error $e(t)$, definido como la diferencia entre una variable de proceso medida (por ejemplo, el desplazamiento lateral de la línea en el frame) y el valor resultante deseado o setpoint (el centro exacto del frame). La señal de control u salida del controlador $y(t)$ se calcula mediante la suma ponderada de tres términos que actúan sobre el error presente, pasado y futuro, expresada matemáticamente como:

$$y(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de}{dt} \quad (3.3)$$

Donde K_p , K_i y K_d representan las constantes o ganancias de sintonización del sistema:

- **Término Proporcional** (K_p): Produce una acción de control directamente proporcional al valor del error actual $e(t)$. Si el error es grande, la respuesta de corrección es fuerte. Una ganancia K_p excesivamente elevada incrementa la velocidad de respuesta del sistema, pero induce oscilaciones e inestabilidad alrededor del setpoint. Si es demasiado baja, el sistema responde con lentitud y puede no alcanzar nunca la corrección necesaria.
- **Término Integral** (K_i): Evalúa la acumulación de los errores pasados a lo largo del tiempo. Su función principal es eliminar el error en estado estacionario, garantizando que el sistema alcance con precisión el valor de consigna exacto. En la navegación de drones, el término integral ayuda a corregir perturbaciones constantes como derivas físicas (viento) o el desequilibrio aerodinámico provocado por asimetrías de peso. No obstante, una acumulación desmedida de este término puede generar un efecto de saturación que vuelve lento el frenado del vehículo.
- **Término Derivativo** (K_d): Analiza la velocidad de cambio del error en el tiempo, proporcionando una estimación del comportamiento futuro del sistema. Actúa esencialmente como un freno amortiguador que atenúa las oscilaciones causadas por el término proporcional. Si el dron se aproxima rápidamente a la línea central, el término derivativo reduce la potencia de giro con antelación para evitar que el dron sobrepase el centro.

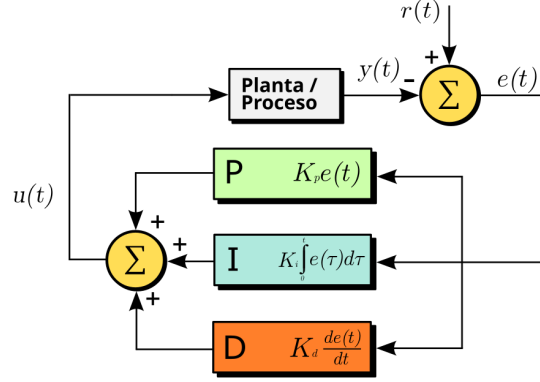


Figura 3.5: Diagrama de un controlador PID [17]

3.5.1. Análisis comparativo de arquitecturas de control (P, PI, PD y PID)

Para comprender la necesidad de una regulación dinámica en el guiado del UAV se debe analizar cómo responde el vehículo ante diferentes configuraciones del lazo cerrado (P, PI, PD y PID) al enfrentarse a un error de trayectoria (desviaciones):

- **Control Proporcional (P)** [$y(t) = K_p e(t)$]: Es la estructura más simple. La señal enviada a los motores es directamente proporcional a la distancia o ángulo que separa al dron de la línea. En la práctica con el DJI Tello, un control puramente proporcional es incapaz de estabilizar el vuelo. Debido a la inercia del cuadricóptero, cuando el controlador intenta corregir la posición, el vehículo adquiere una velocidad que no se frena al llegar al setpoint (por ejemplo, $e(t) = 0$). Esto provoca que el dron sobrepase la línea continuamente, entrando en un estado de oscilación continuada, como se puede ver en el ejemplo de la Figura 3.6a.
- **Control Proporcional-Integral (PI)** [$y(t) = K_p e(t) + K_i \int e(\tau) d\tau$]: El término integral acumula las desviaciones en el tiempo para eliminar el error persistente (por ejemplo, si el dron se queda desplazado unos centímetros a un lado debido a una corriente de aire lateral). Sin embargo, añadir la componente integral a un sistema subamortiguado como un UAV en interiores empeora drásticamente la situación descrita en el control P. La acumulación del error pasado introduce un retraso que amplifica las oscilaciones y satura los motores, haciendo que el dron dibuje trayectorias sinusoidales cada vez más amplias hasta salirse por completo de la pista.
- **Control Proporcional-Derivativo (PD)** [$y(t) = K_p e(t) + K_d \frac{de}{dt}$]: Esta configuración prescinde de la componente integral y añade la acción derivativa, la cual actúa de forma predictiva analizando la velocidad a la que el dron se aproxima o aleja de la trayectoria ideal. El término K_d actúa como un amortiguador. Si el dron se dirige rápidamente hacia el centro de la línea, la derivada del error cambia de signo y frena el movimiento con antelación. Esta arquitectura aporta una estabilidad y rapidez que un control PI o P no podrían alcanzar sin la componente derivativa [18]. Observando la Figura 3.6c vemos como es la más rápida en estabilizar su error de centrado sin realizar oscilaciones indeseadas. Por esto

es que ha sido la arquitectura implementada en este proyecto en ambos lazos de control, tanto en el lazo de *Yaw* como el de *Roll*.

- **Control Proporcional-Integral-Derivativo (PID)** $[y(t) = K_p e(t) + K_i \int e(\tau) d\tau + K_d \frac{de}{dt}]$: Combina las ventajas de las tres acciones: la rapidez del término proporcional, la amortiguación del derivativo y la corrección del integral. Teóricamente es el sistema perfecto ante perturbaciones externas continuas (como ráfagas de viento en exteriores). No obstante, para entornos controlados de interior y con tiempos de vuelo reducidos, la introducción de una ganancia $K_i > 0$ añade una complejidad de innecesaria y el riesgo de retrasar la respuesta del Tello en las transiciones rápidas de curvas cerradas o bifurcaciones. Si observamos la Figura 3.6d vemos que es parecida a la respuesta de un control PD, pero interviene una oscilación que se atenúa con el tiempo añadiendo retrasos en la orientación y centrado del dron sobre la línea a seguir.

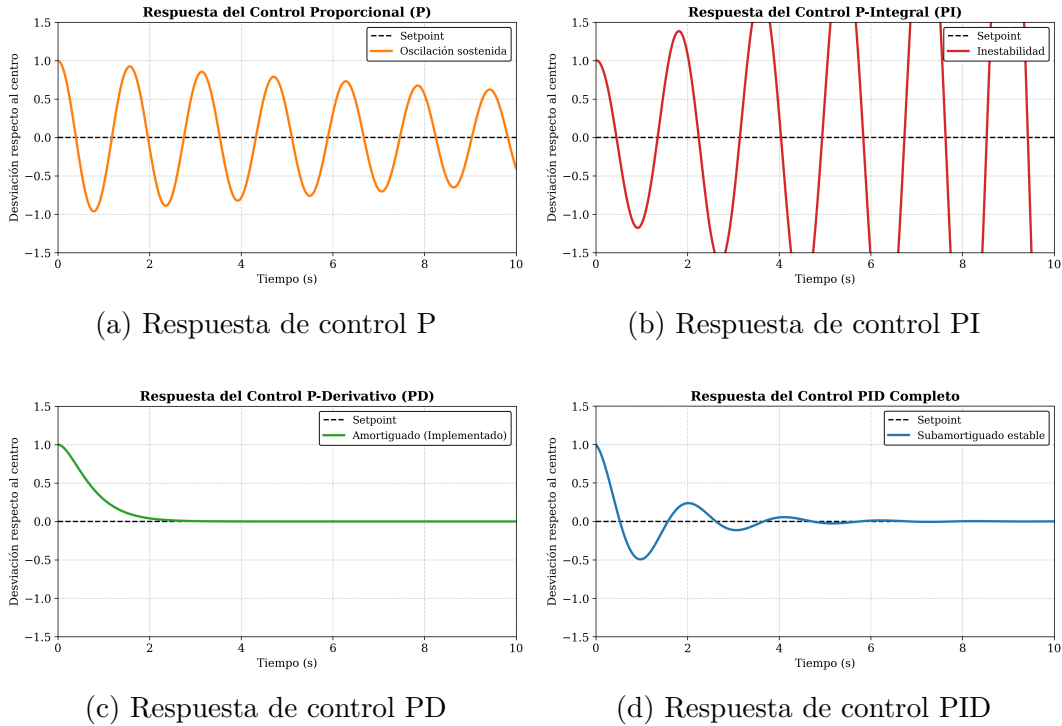


Figura 3.6: Comparación de respuestas de arquitecturas de control P, PI, PD y PID.

3.5.2. Técnicas de sintonización y cálculo de coeficientes del controlador

El rendimiento y la estabilidad de los lazos de control propuestos dependen del cálculo de sus ganancias dimensionales (K_p , K_i y K_d). Para determinar estos parámetros, la literatura ofrece diferentes metodologías:

- **Métodos analíticos:** Se fundamentan en ecuaciones diferenciales o funciones de transferencia, precisando de un modelado matemático sumamente preciso del entorno.

- **Método de Ziegler-Nichols:** Es una de las técnicas de sintonización industrial más extendidas. Basado en la respuesta temporal del sistema, su enfoque en lazo cerrado consiste en anular las componentes integral y derivativa ($K_i = 0$, $K_d = 0$) e incrementar progresivamente la ganancia proporcional hasta alcanzar una oscilación sostenida. A partir de esta ganancia crítica (K_{cr}) y del periodo de oscilación resultante (P_{cr}), se aplican tablas estandarizadas para calcular los parámetros del regulador [19].
- **Métodos heurísticos y ajuste manual:** Consisten en una estrategia de ensayo y error guiada por el comportamiento empírico del dron. El proceso se inicia sintonizando la acción proporcional (K_p) desde cero hasta que el sistema reaccione con rapidez ante el error. Posteriormente, se introduce y eleva el término derivativo (K_d) para actuar como amortiguador, suavizando la aproximación al *setpoint* y eliminando las sobrecorrecciones. En el caso concreto de este proyecto se descarta el uso de una componente integral (K_i) debido a la inexistencia de perturbaciones externas como el viento.

3.5.3. Aplicación a la cinemática del cuadricóptero

Para el guiado del DJI Tello sobre una trayectoria bidimensional en el suelo, el control no puede limitarse a un único parámetro, requiriendo el funcionamiento en paralelo de dos lazos de control independientes que mitiguen dos tipos de errores geométricos distintos:

- **Lazo de Guiñada (*Yaw*):** Regula la velocidad de rotación sobre el eje vertical Z del dron. El error de entrada se calcula a partir de la desviación angular producto del corte de la línea en los bordes del frame y el corte horizontal de la línea a la altura con respecto del eje central del dron en el frame. Este PID se encarga de que el dron rote sobre su propio centro para alinearse en la misma dirección que la pista, manteniéndose paralelo a las rectas y encarando las curvas de forma suave.
- **Lazo de Alabeo (*Roll*):** Regula el desplazamiento lateral o traslación hacia la izquierda o derecha sobre el eje X del dron. El error de entrada se define como la distancia en píxeles (desplazamiento lateral) entre el centroide de la línea negra y el centro del frame. Este PID actúa de manera simultánea al Yaw, corrigiendo las desviaciones laterales y asegurando que, aunque el dron esté orientado correctamente hacia adelante, su plano vertical se sitúe exactamente encima del eje de la cinta adhesiva.

Parte II

Desarrollo

4. Hardware y Entorno Físico

Este capítulo describe en detalle las modificaciones mecánicas y ópticas realizadas sobre el cuadricóptero DJI Tello para reorientar su campo de visión hacia el plano del suelo. Asimismo, se expone el diseño, los materiales y las restricciones geométricas del circuito de pruebas desarrollado para la validación del sistema, y se analizan los problemas de estabilidad espacial derivados de la naturaleza del entorno físico junto con las soluciones de hardware implementadas para mitigarlos.

4.1. Modificación del sistema de visión del DJI Tello

El DJI Tello está diseñado originalmente para capturar imágenes del entorno en un plano frontal, integrando una cámara fija en su chasis que carece de un sistema móvil de orientación. Dado que el objetivo principal de este trabajo es el seguimiento de una trayectoria dispuesta sobre el suelo, la cámara frontal resulta inservible en su configuración de fábrica. Para solventar esta limitación sin realizar modificaciones destructivas ni alterar los componentes electrónicos internos del dron, se optó por un enfoque basado en la desviación del eje óptico mediante reflexión.

4.1.1. Diseño y evolución de la pieza 3D

La solución consistió en la modificación e impresión 3D de una pieza de acople que se monta sobre la parte superior del chasis del dron. Esta pieza actúa como soporte para un pequeño espejo plano que intercepta el campo de visión de la cámara frontal y lo desvía verticalmente hacia el suelo, proporcionando una vista pseudocenital de la pista. El proceso de desarrollo de este soporte óptico requirió un diseño iterativo debido a las necesidades de anticipación visual.

Prototipo inicial (Inclinación de 45°)

En la primera fase del proyecto, se descargó y fabricó un modelo digital [20] con una inclinación fija de 45 grados respecto al eje horizontal de la cámara. A una altura de vuelo de referencia de 60 centímetros, esta configuración proporcionaba una visión puramente cenital, permitiendo cubrir un área simétrica en el suelo de 20 centímetros hacia adelante y 20 centímetros hacia atrás respecto a la proyección vertical del centro del dron. Durante las pruebas empíricas se constató que la información visual de los 20 centímetros posteriores carecía de utilidad para el algoritmo de guiado. Además, al disponer de solo 20 centímetros de visión frontal, la capacidad de anticipación del dron ante cambios de trayectoria o curvas era muy reducida, lo que obligaba al vehículo a desplazarse a velocidades muy bajas para disponer de suficiente tiempo de frenada.

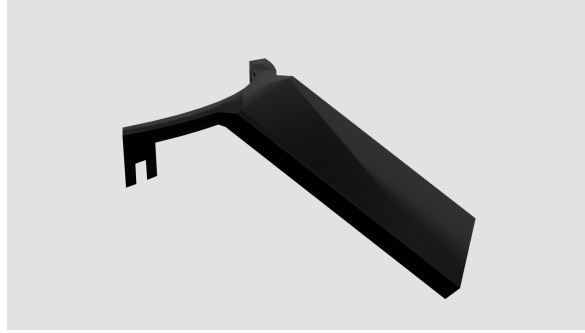


Figura 4.1: Render 3D del prototipo inicial en el software *Blender*

Prototipo optimizado (Inclinación de 50°)

Con el objetivo de incrementar la distancia de anticipación visual y permitir mayores velocidades de traslación, se procedió a modificar el modelo original utilizando el software de modelado Blender. Se alteró la geometría del soporte para modificar el ángulo del espejo a 50 grados. Con esta sutil variación angular, manteniendo la altura de referencia de 60 centímetros, el campo de visión útil se desplazó significativamente hacia el frente: el dron pasó a visualizar 40 centímetros hacia adelante y únicamente 13 centímetros hacia atrás. Este incremento del 100 % en la distancia de visión frontal otorgó una mayor libertad al sistema de control para regular la velocidad en rectas y anticipar con suficiente margen las transiciones hacia curvas e intersecciones.

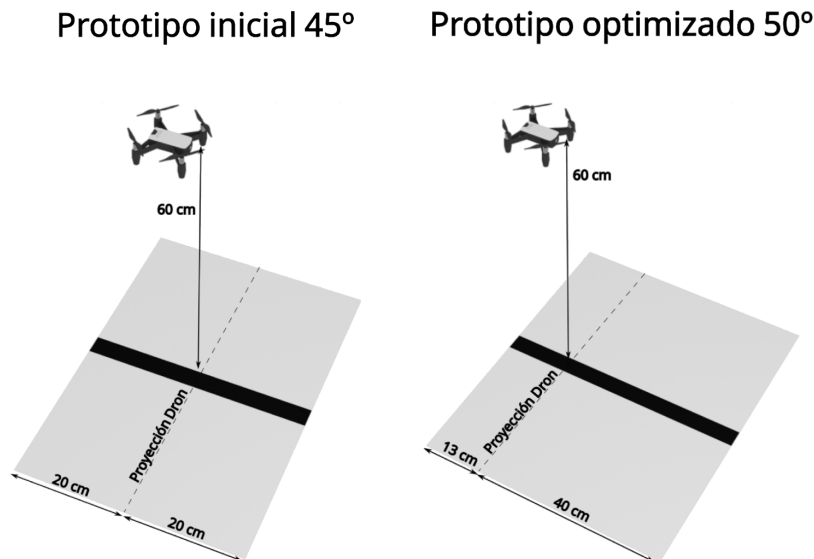


Figura 4.2: Comparativa de visiones con ambos prototipos

4.1.2. Especificaciones técnicas del conjunto óptico

El conjunto mecánico añadido debe cumplir con estrictas tolerancias de peso para no sobrecargar los motores del DJI Tello ni comprometer la dinámica de vuelo.

El sistema óptico final está compuesto por:

1. Un soporte de polímero ligero impreso en 3D con un peso de 3.57 gramos.
2. Un espejo plano con unas dimensiones de 40 milímetros de largo, 36 milímetros de ancho, un espesor ultrafino de 1 milímetro y 6.43 gramos de peso.
3. Un soporte de goma en la parte inferior de cada una de las 4 patas del dron, añadiendo así 14 milímetros de altura y 0.74 gramos de peso, necesarios para salvaguardar el soporte de polímero y el espejo de aterrizajes forzosos.



Figura 4.3: Dron DJI Tello con las modificaciones añadidas

El peso conjunto del soporte de polímero, el espejo de vidrio y los soportes de goma es menor a 11 gramos. Esta carga útil se sitúa dentro de las capacidades admisibles del dron, permitiendo el despegue y la maniobrabilidad sin provocar un calentamiento excesivo de los motores ni reducir drásticamente los tiempos de vuelo de la batería.

Desde un punto de vista teórico, el diseño ideal consistiría en una pieza óptica capaz de reflejar exclusivamente la región del suelo situada por delante del dron, eliminando por completo los 13 centímetros de visión posterior que siguen resultando inservibles. Sin embargo, debido al campo de visión nativo de 82.6° de la cámara del Tello para lograr este propósito, se requeriría un soporte considerablemente más largo y adelantado. Esta geometría desplazaría el espejo hacia el frente, generando un gran contrapeso delantero. Esto alteraría críticamente el centro de gravedad del cuadricóptero, obligando a los motores frontales a trabajar a su máxima potencia para compensar el cabeceo (*pitch*), desestabilizando por completo el vuelo automático y haciendo inviable la propuesta.

4.2. Diseño del entorno de pruebas

Para la validación del sistema en un entorno controlado que simule las condiciones operativas de una planta industrial, se procedió a la construcción de un circuito físico específico. En este integran tanto los elementos de guiado, como los sistemas de señalización lumínica requeridos por el algoritmo. No obstante, se restringen ciertos aspectos físicos del circuito para facilitar una lógica más simple en el software.

4.2.1. Materiales y estructura de la pista

La base del circuito consiste en varios trozos de tela blanca de grandes dimensiones que ha sido cortada y cosida para adaptarse a la morfología del entorno de pruebas. Sobre esta superficie textil blanca, que actúa como un fondo uniforme de alto contraste y textura homogénea, se ha dispuesto la trayectoria de seguimiento utilizando cinta adhesiva negra. Con el fin de proporcionar un ancho de vía suficiente para facilitar la segmentación e impedir que el dron pierda la trayectoria durante las oscilaciones laterales, la línea principal se construyó con un ancho equivalente a dos tiras de cinta adhesiva juxtapuestas.

Fijada en el eje central de esta trayectoria, justo en medio de las dos tiras de cinta negra, se instaló una tira de diodos LED encapsulados. Esta tira lumínica tiene la capacidad de emitir diferentes colores y es controlada en la implementación actual mediante un mando emisor de infrarrojos, permitiendo modificar en tiempo real el color de los diodos para enviar comandos al dron durante las pruebas de vuelo. Si bien la integración mediante control por infrarrojos es efectiva para esta fase de desarrollo, la configuración industrial óptima implicaría una conexión cableada o una interfaz de red automatizada conectada directamente al ordenador central para sincronizar dinámicamente las señales de la planta con la trayectoria del dron, evitando así las perturbaciones en el espacio inalámbrico que deseamos evitar en estos entornos industriales.

4.2.2. Complejidad y restricciones geométricas del trazado

El diseño del circuito experimentó una evolución paralela a la madurez de los algoritmos de control. Inicialmente, las pruebas se limitaron a trayectorias en línea recta y curvas para calibrar las ganancias básicas de ambos controladores PID (Guiñada y Alabeo). Posteriormente, se añadieron bifurcaciones e intersecciones para comprobar el correcto funcionamiento de las acciones mediante las señales luminosas de la tira LED. Finalmente, la pista se dotó de una estructura de red compleja orientada a simular la distribución de una planta de fabricación real, pero restringido por el espacio cerrado donde se realizan las pruebas.

Para garantizar la viabilidad del seguimiento por visión computacional y respetar las capacidades del DJI Tello, el trazado se diseñó bajo estrictas restricciones geométricas:

- **Radio de curvatura uniforme:** Todas las curvas del circuito presentan un radio de curvatura fijo de exactamente 90 centímetros. Esta homogeneidad geométrica surge ante la necesidad de paliar la complejidad del sistema de control PID, siendo necesario un menor ajuste de ellos para el mismo radio en todas las curvas del circuito.
- **Limitación de bifurcaciones e intersecciones:** El circuito está concebido como un mapa estructurado de forma que ninguna intersección o bifurcación puede disponer de más de dos opciones de salida o de entrada simultáneas. Esta restricción de diseño simplifica la lógica binaria de toma de decisiones del software: ante una bifurcación, el dron solo se enfrentará a la dualidad de escoger entre el camino de la izquierda o el de la derecha, evitando ambigüedades que requerirían sistemas de inspección visual más complejos.

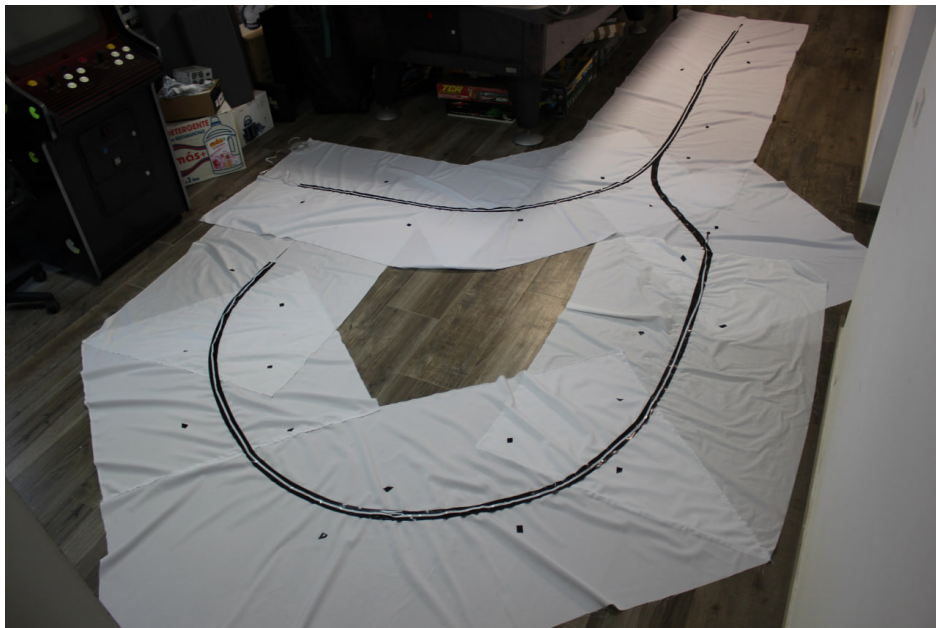


Figura 4.4: Circuito de pruebas en un espacio cerrado

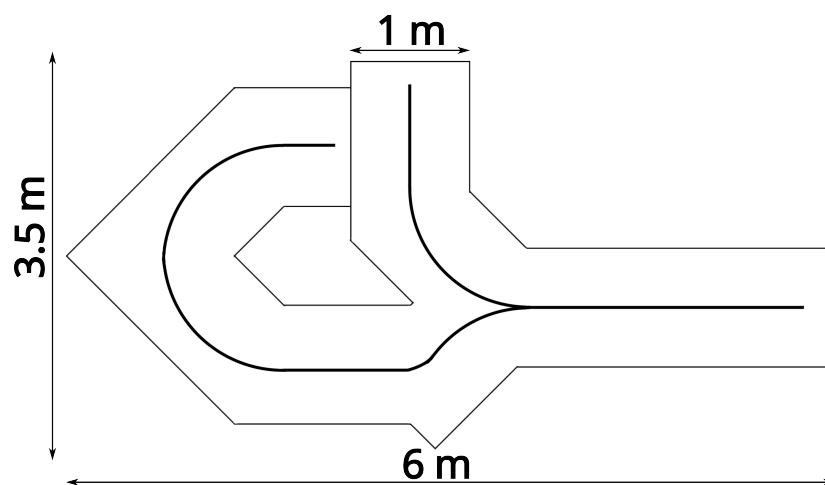


Figura 4.5: Plano del circuito de pruebas

4.3. Problemas de estabilidad y soluciones de hardware

Uno de los mayores desafíos técnicos detectados durante las primeras pruebas de vuelo autónomo estuvo relacionado con la severa inestabilidad posicional del dron al desplazarse a lo largo de las secciones rectilíneas del circuito.

4.3.1. Fallo del Sistema de Posicionamiento por Visión (VPS)

Como se ha expuesto en el Contexto teórico (3), el DJI Tello depende críticamente de su sensor inferior de flujo óptico para mantener su posición estable en el espacio y contrarrestar las derivas de viento o las asimetrías de empuje de los rotores.

Al introducir el dron en el entorno de pruebas, cuyo fondo está constituido por una tela blanca completamente homogénea, lisa y libre de texturas, el sensor de flujo óptico inferior quedaba anulado por la falta de contraste o texturas. Al no detectar puntos de interés que rastrear en el fondo blanco, el sistema de estabilización interna del dron sufría fallos continuos, lo que provocaba que el vehículo experimentara movimientos impredecibles, comprometiendo la consistencia de la navegación, incluso volando sobre una línea recta.

4.3.2. Implementación de puntos de anclaje visuales

Para solucionar este problema sin alterar la lógica de programación ni añadir sensores adicionales que incrementaran el peso del dron, se diseñó una solución pasiva basada en la alteración del entorno. Se procedió a la colocación estratégica de trozos de cinta adhesiva y pegatinas negras de diversas formas y posiciones a lo largo del circuito, dispuestas de manera adyacente a la línea de guía principal en los tramos rectos.

Estas pegatinas actúan como puntos de anclaje visuales. Aunque debido al carácter cerrado del firmware propietario de DJI no es posible confirmar a bajo nivel el mapa de características internas que genera el procesador de vuelo, los resultados experimentales confirmaron la efectividad de la medida. Al realizar pruebas sobrevolando el mismo recorrido se pudo comprobar que la inclusión de estos puntos de anclaje estabilizó de forma notable el comportamiento del dron en las rectas, reduciendo las oscilaciones a valores mínimos.

Asimismo, para evitar que estas pegatinas de estabilización interfirieran negativamente con el algoritmo de guiado de la cámara superior, el tamaño superficial de estos puntos se diseñó de forma restrictiva para que pudiera ser fácilmente filtrado y descartado por tamaño en la etapa de binarización del software (5.4.3), como se detallará más adelante.

5. Sistema de Visión por Computador

El núcleo del sistema de navegación autónoma reside en su capacidad para percibir e interpretar el entorno físico a través de secuencias de vídeo en tiempo real. Dado que toda la carga lógica de guiado, estabilización y toma de decisiones se apoya en la información extraída de los fotogramas, es crucial diseñar un *pipeline* de procesamiento visual que combine un alto nivel de rigor y alta eficiencia temporal. El procesamiento debe realizarse a una tasa lo suficientemente elevada como para ser capaz de completar la ejecución de todas las tareas sin sobrepasar el tiempo en recibir el siguiente fotograma válido.

Este capítulo detalla el desarrollo, fundamentación matemática e implementación del sistema visual en la estación de tierra o servidor. Se describen los métodos de adquisición y descarte de fotogramas, las técnicas de preprocesamiento geométrico, el subsistema de calibración y detección cromática de señales LED mediante un modelo estadístico gaussiano, y los procedimientos de segmentación morfológica orientados a aislar la trayectoria. También se aborda el análisis geométrico de las figuras binarizadas mediante el cálculo de indicadores, el cálculo trigonométrico del desplazamiento lateral (*roll*) apoyado en el sensor de altura y la integración de marcadores ArUco como base del futuro mapa topológico.

5.1. Adquisición y gestión de frames

El bucle principal del programa se inicia con la solicitud explícita de un fotograma a la plataforma de vuelo a través de las funciones de captura de la librería *djitellopy* [21]. La velocidad e integridad de este proceso son críticas, ya que cualquier retraso en la captura de la imagen o una interpretación errónea se traduce de forma directa en una ejecución incorrecta del pipeline completo.

5.1.1. Control de flujo y gestión de nulos

Debido a las fluctuaciones aleatorias en el canal de comunicación inalámbrico Wi-Fi, el hilo de recepción de vídeo puede sufrir microcortes o retrasos en la entrega de paquetes de datos. Para evitar que el hilo principal del algoritmo intente procesar una estructura de datos vacía, se ha implementado un mecanismo de control de flujo:

1. **Validación de existencia:** Tras solicitar el fotograma al búfer de vídeo, el sistema verifica de forma inmediata si el objeto recibido es nulo.
2. **Espera activa controlada:** Si el frame resulta nulo, el algoritmo detiene su ejecución de manera controlada durante 1 milisegundo antes de volver a llamar al búfer. Esta pequeña pausa libera ciclos de la CPU.

5.1.2. Algoritmo propio de comparación y descarte de duplicados

Una vez garantizada la validez del fotograma, se presenta un segundo problema: la redundancia. El DJI Tello retransmite vídeo a una tasa fija de 30 fps; sin embargo, debido a la latencia de red o a una rápida ejecución del pipeline de procesamiento, es bastante frecuente que el algoritmo principal solicite un nuevo frame antes de que la cámara haya refrescado la imagen.

Procesar dos veces seguidas exactamente el mismo fotograma supone un gasto innecesario de recursos computacionales, además de una posible infracorrección o sobrecorrección sobre la misma imagen procesada. Para resolver este inconveniente, se diseñó un algoritmo propio de comparación de fotogramas secuenciales. Aunque la librería *NumPy*¹ implementa funciones de comparación matricial altamente optimizadas en código C++, estas evalúan por defecto la totalidad de la matriz, lo que introduce un sobrecoste de computación cuando las imágenes son diferentes. El método jerárquico desarrollado en este proyecto optimiza la detección de cambios basándose en la premisa de que dos fotogramas consecutivos de un dron en movimiento diferirán rápidamente en múltiples puntos:

1. **Evaluación mono-píxel** (Nivel 1): El algoritmo inspecciona un único píxel situado en una coordenada central estratégica, definida en la fila $y = \frac{H}{4}$ y la columna $x = \frac{W}{2}$. Si los valores de sus canales BGR son distintos entre el frame actual y el anterior, se asume de forma inmediata que la imagen ha cambiado, dando por finalizada la función y devolviendo un resultado negativo de manera inmediata.
2. **Evaluación multi-píxel dispersa** (Nivel 2): En caso de que el primer píxel sea idéntico, el algoritmo evalúa un patrón disperso de 6 píxeles adicionales distribuidos uniformemente a lo largo de la misma horizontal ($y = \frac{H}{4}$) mediante un muestreo lineal espaciado ($np.linspace(0, w - 1, 6)$). Si cualquiera de estos puntos muestra una variación, se dictamina que el fotograma es distinto y se rompe la ejecución.
3. **Evaluación matricial completa** (Nivel 3): Únicamente si los 7 píxeles de control resultan ser rigurosamente iguales, el algoritmo recurre a la comprobación de la matriz completa. Si se confirma la igualdad de fotogramas, el sistema suspende la iteración durante 1 milisegundo y vuelve a solicitar un fotograma al búfer.

Las pruebas empíricas demostraron la alta eficiencia de este enfoque jerárquico, donde el tiempo de computación es casi 10 veces menor en comparación con la función de Numpy cuando los fotogramas son diferentes, y aproximadamente un 30 % más lento en fotogramas iguales.

¹NumPy - <https://numpy.org/>

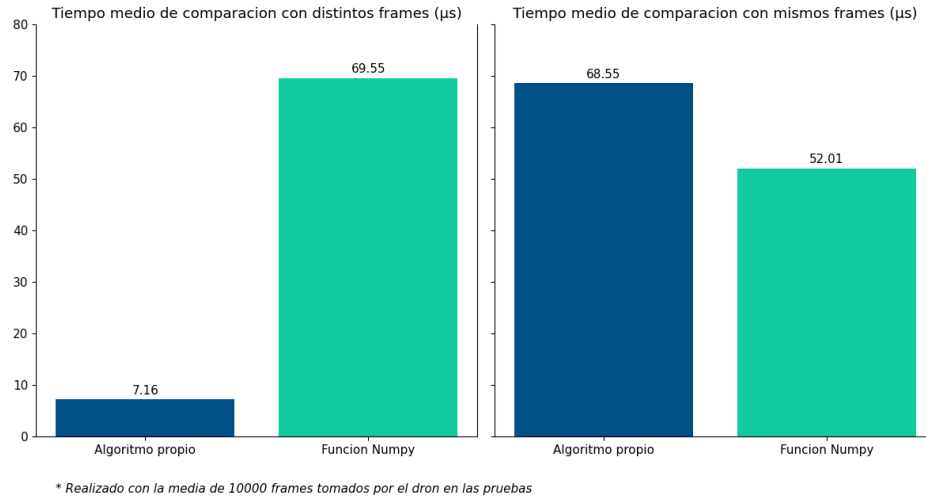


Figura 5.1: Gráfico de barras comparativo entre el tiempo medio de ejecución del algoritmo propio y la función de NumPy (*np.array_equal*)

5.2. Detección del estado luminoso

La tira de diodos LED dispuesta sobre el circuito actúa como el canal de comunicación unidireccional mediante el cual el entorno dicta las órdenes operativas al dron. El subsistema visual implementado debe aislar estas fuentes de luz y clasificar su color con total fiabilidad. Asimismo, debe poder permitir la calibración para su uso fuera del entorno de pruebas fijado.

5.2.1. Aislamiento en el espacio HSV

Para independizar la detección de las condiciones de iluminación ambiental de la sala, el fotograma a color sin preprocesar se convierte al espacio de color HSV. Esta detección se hace con el fotograma original, puesto que una modificación en el tamaño de la imagen reduce la información luminosa microscópica, ocasionando pérdidas importantes en la información visual necesaria para la detección de focos de luz en la imagen.

El aislamiento de los píxeles pertenecientes a los diodos LED se realiza mediante la aplicación de un filtro de umbralización estricto sobre los canales de Saturación (S) y Valor (V). Específicamente, para que un píxel sea considerado como un candidato válido a formar parte de la señal lumínica, debe cumplir simultáneamente los siguientes criterios:

$$\text{Saturación } (S) > 30 \quad \wedge \quad \text{Valor } (V) > 190$$

Este filtro aprovecha que los diodos LED en funcionamiento emiten una luz con una intensidad luminosa extremadamente alta (alto valor de V) y un color muy vivo

(saturación superior a 30). La imposición de estos límites discrimina de forma automática los puntos pequeños procedentes de destellos de la luz ambiente sobre la cinta adhesiva o imperfecciones de la tela blanca, aislando exclusivamente los focos de emisión activa.

5.2.2. Algoritmo de calibración probabilística por curvas gaussianas

Debido a que las componentes del canal de Matiz (Hue, H) pueden sufrir variaciones físicas dependiendo del voltaje de la tira LED, la degradación óptica a través del espejo o cambios en la iluminación del entorno de pruebas, resulta desaconsejable utilizar fronteras de color fijas codificadas en el software. En su lugar, se ha diseñado un protocolo de calibración automática previa al vuelo:

1. **Muestreo dinámico:** Al iniciar el sistema con la bandera de calibración activa, el programa solicita secuencialmente al operador que configure la tira LED en un color determinado durante una ventana temporal fija (ajustable antes de la ejecución del programa). Los valores del canal H correspondientes a los píxeles brillantes que superan el filtro de intensidad se almacenan en un vector histórico. Tras esa ventana temporal existe una pausa de calibración (ajustable antes de la ejecución del programa) donde se indica el siguiente color que el operador deberá fijar, sin almacenar valores en el vector histórico. Después de la ventana de pausa, comenzará la de calibración. Esto sucederá para cada uno de los 6 colores existentes en la paleta cromática de colores seleccionada.
2. **Tratamiento de la circularidad del color:** El canal de Matiz en OpenCV se distribuye en un rango cerrado de $[0, 179]$, lo que provoca que el color rojo aparezca fragmentado en ambos extremos del espectro (entre 0-10 y 170-179). Para evitar que este fenómeno arruine el cálculo de los estadísticos lineales, el algoritmo aplica una transformación condicional sobre las muestras del color rojo, desplazando los valores del extremo superior hacia el rango negativo:

$$\text{si } H > 90 \Rightarrow H_{\text{ajustado}} = H - 180$$

De este modo, la distribución del rojo se unifica de manera continua en torno al valor central de 0.

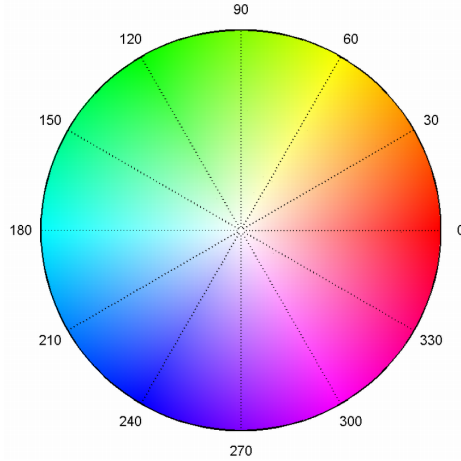


Figura 5.2: Representación numérica canal Hue (0-360) ²

3. **Modelado matemático e intersección de gaussianas:** A partir de los datos limpios de cada color, se calcula su media matemática (μ) y su desviación estándar (σ). Para hallar la frontera de decisión aproximada entre dos colores adyacentes del espectro, se implementa una función que resuelve analíticamente el punto de cruce ponderado por la dispersión de ambas campanas de Gauss:

$$x = \frac{\mu_1\sigma_2 + \mu_2\sigma_1}{\sigma_1 + \sigma_2} \quad (5.1)$$

Si bien este modelado para encontrar las fronteras entre ambas distribuciones no encuentra el punto óptimo de decisión, es suficiente puesto que el nivel de cómputo requerido por la función es ínfimo, balanceando precisión y bajo nivel de computación requerido.

Este procedimiento genera un conjunto de umbrales dinámicos (*boundaries*) que dividen el espacio de Matiz con alta precisión adaptativa. Durante el vuelo, para cada contorno LED detectado, se extrae el histograma de su región delimitadora y se suman los píxeles contenidos dentro de las fronteras inter-gaussianas calculadas (véase la Figura 5.3). El color con el valor máximo en la votación matemática determina la identidad cromática del contorno.

En una calibración podemos observar (véase la Tabla 5.1) como algunos colores como el cian tienen un valor post-calibración semejante a su valor teórico. Asimismo, colores como el verde o el rosa tienen una distancia considerable entre su valor teórico y su valor de calibración, resultando en falsos positivos que merman la estabilidad del sistema de calibración.

²Harmonizing Colours - <https://svijaykoushik.github.io/blog/2019/03/03/harmonizing-colours/>

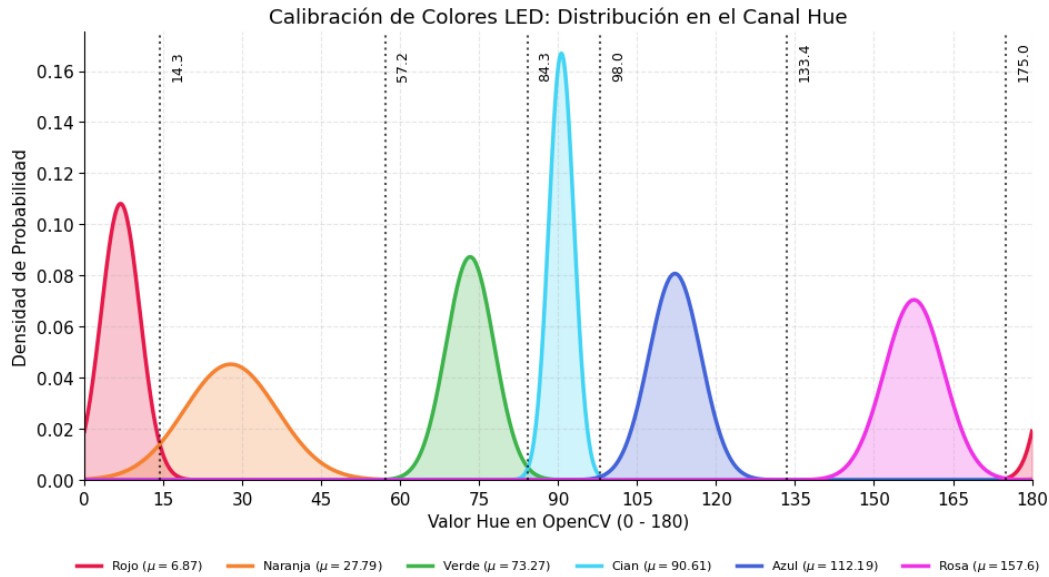


Figura 5.3: Ejemplo de calibración de fronteras de curvas Gaussianas para los 6 colores

Tabla 5.1

Comparativa entre umbrales teóricos y después de la calibración.

Color	Valor Hue teórico (0-180)	Valor Hue resultado de la calibración
Rojo	0	7
Naranja	25	28
Verde	60	73
Cyan	90	91
Azul	120	112
Rosa	147	158

5.2.3. Mitigación de la asincronía temporal mediante filtrado de moda (Cola FIFO)

Un problema técnico detectado en la tira de LEDs comercial es que sus diodos no realizan la transición cromática de forma instantánea a lo largo del circuito. Al enviar una orden de cambio de color, existe una ventana temporal de decenas de milisegundos en la que coexisten dos colores distintos en diferentes tramos de la tira dentro del mismo fotograma, lo que podría inducir a una detección errónea del estado lumínico en un fotograma al realizar un cambio de color. Para mitigar este fenómeno de transición

lumínica, se implementó un mecanismo de suavizado temporal basado en una cola de tipo *FIFO* (First In, First Out) de 5 elementos:

1. En cada iteración del bucle, el algoritmo extrae el color mayoritario presente en el fotograma actual.
2. Este color se introduce en la cola de memoria, descartando el elemento más antiguo.
3. El color definitivo que se devuelve al sistema de navegación no es el del frame actual, sino el resultado de calcular la moda estadística de los 5 elementos de la cola.

Este filtro introduce un retraso insignificante de 2 frames (aproximadamente 66 milisegundos), pero aporta una robustez absoluta frente a frames de transición o lecturas erróneas aisladas.

5.2.4. Paleta cromática de comandos LED

Las funciones de navegación asociadas a cada uno de los colores de la tira LED son las siguientes:

- **None** (Sin emisión): Mantener de forma inercial la última acción de guiado registrada antes del apagado.
- **Rojo**: Parada absoluta del vehículo (*stop*). Se fuerza la velocidad de avance a 0, deteniendo el dron en vuelo semi-estacionario, dado que los errores de Guiñada y Alabeo seguirían corriéndose, posicionando el dron de manera correcta en la línea.
- **Verde**: Indicador de dirección en bifurcación hacia la izquierda.
- **Azul**: Indicador de dirección en bifurcación hacia la derecha.
- **Rosa**: Comando de finalización de vuelo o aterrizaje controlado.
- **Naranja**: Inversión de la marcha cinemática (vuelo hacia atrás).
- **Cian**: Maniobra de media vuelta inmediata. El dron ejecuta un giro puro de 180 grados sobre su eje vertical y limpia la cola *FIFO* para evitar ejecuciones duplicadas debido a la latencia de la tira.

5.3. Preprocesamiento de imágenes

Una vez que se dispone de un fotograma válido y diferenciado de su predecesor, la matriz de píxeles cruda debe someterse a una serie de transformaciones para optimizar los tiempos de cómputo de las etapas posteriores sin que esto incurra en una pérdida de información para tareas posteriores.

5.3.1. Inversión de la imagen

Como se ha detallado en el diseño del hardware, el dron captura el suelo a través de la reflexión de un espejo montado en un soporte a 50 grados. Este fenómeno óptico introduce una inversión en la imagen capturada por el sensor frontal del dron. Concretamente, la reflexión sobre el espejo inclinado transpone el eje vertical de la imagen.

Para restablecer la coherencia espacial del entorno se aplica una operación de volteo sobre el eje X. Esta transformación geométrica corrige la orientación de la marcha, simulando de manera precisa una cámara cenital orientada en el sentido de avance del cuadricóptero, facilitando el diseño de algoritmos, así como el entendimiento y depuración de los mismos.

5.3.2. Reducción de escala

El fotograma original es entregado por el dron con una resolución nativa de 960x720 píxeles. Someter este volumen de datos a operaciones matriciales complejas como la umbralización adaptativa o las convoluciones morfológicas de gran tamaño comprometería la viabilidad del control en tiempo real.

Con el objetivo de maximizar la eficiencia temporal, se realiza un reescalado dimensional reduciendo las dimensiones de la imagen a una resolución fija de 640x480 píxeles, conservando así su relación de aspecto entre el ancho y la altura del fotograma. Esta reducción disminuye la carga computacional en más de un 55 %. Dado que las características geométricas que se pretenden extraer de la pista (la línea de cinta adhesiva) son de carácter macroscópico, esta pérdida de resolución espacial no degrada la precisión de la segmentación ni compromete la calidad de la información necesaria para el guiado.

5.4. Segmentación y binarización

El aislamiento de la línea de guiado negra sobre el fondo de tela blanca requiere un pipeline de binarización robusto y eficiente. Este debe ser capaz de adaptarse a distintos niveles de luminosidad, eliminar los elementos innecesarios y las marcas físicas de estabilidad sin alterar la morfología macroscópica del camino principal.

5.4.1. Transformaciones y suavizado de ruido

El primer paso de esta etapa consiste en la conversión del fotograma corregido a escala de grises para trabajar con una matriz de un solo canal de intensidad. Inmediatamente después, se aplica un filtro de desenfoque gaussiano utilizando un núcleo cuadrado o kernel de dimensiones 5x5 píxeles con una desviación estándar auto-calculada. Este filtro realiza una convolución matemática sobre la imagen que difumina los bordes de alta frecuencia [22], suprimiendo el ruido visual a pequeña escala provocado por

las texturas del cosido de la tela, pequeñas pelusas acumuladas en el suelo o destellos puntuales reflejados en la superficie plástica de la cinta negra.

5.4.2. Umbral adaptativo local

Para realizar la separación efectiva de la línea frente al fondo bajo condiciones de iluminación heterogénea, se ejecuta un algoritmo de umbralización adaptativa local por media. Los parámetros de este filtro se sintonizaron empíricamente para ofrecer la máxima estabilidad en condiciones comunes para funcionar dentro del entorno de pruebas fijo:

- **Tamaño de bloque** (101×101 píxeles): Se seleccionó una ventana de vecindad amplia. Al emplear un bloque de 101 píxeles, el algoritmo abarca una región significativamente mayor que el ancho de la línea de cinta adhesiva doble, lo que garantiza que el cálculo de la media local no quede saturado por el color negro de la propia trayectoria y mantenga la capacidad de discriminar los bordes de la pista.
- **Constante de ajuste** ($C = 35$): Este valor actúa como un sesgo que se resta a la media calculada en la vecindad. Un valor de 35 asegura que solo las regiones que sean notablemente más oscuras que el promedio local se transformen en píxeles de interés, garantizando que las sombras tenues o las arrugas de la tela blanca sean clasificadas correctamente como fondo y totalmente descartadas.

5.4.3. Operaciones morfológicas y máscaras aisladas

A pesar de la eficacia del umbral adaptativo, la imagen binaria resultante puede presentar imperfecciones estructurales. Para limpiar la imagen de forma sólida, se aplican dos operaciones morfológicas consecutivas utilizando elementos estructurantes rectangulares:

1. **Apertura morfológica:** Con un kernel de 3×3 píxeles y 1 iteración, orientada a eliminar filamentos aislados o motas de polvo sin llegar a eliminar partes estructurales de la cinta adhesiva.
2. **Cierre morfológico:** Con un kernel de 9×9 píxeles y 1 iteración. El propósito de este cierre es fusionar de forma sólida cualquier micro-fisura interna o discontinuidad en el negro de la cinta adhesiva provocada por reflejos de los LEDs centrales, garantizando la homogeneidad de la trayectoria.

Tras el filtrado, se extraen los contornos externos de la matriz resultante de los pasos anteriores. El circuito contiene de forma deliberada dos tipos de elementos negros: la línea de guiado doble y las pegatinas pequeñas distribuidas en las rectas como puntos de anclaje para el flujo óptico del hardware. Para evitar que el algoritmo confunda una pegatina de estabilidad con el camino a seguir, se implementa un filtro de selección por tamaño: se evalúa el área de cada contorno y se establece un umbral de área estricto de 1.500 píxeles. Aquellos que superen este umbral se consideran componentes válidos de la pista.

Como innovación de robustez del código, para cada contorno válido se genera de manera aislada una matriz independiente vacía sobre la cual se dibuja la figura rellena. Esta estrategia da origen a una lista de imágenes binarias independientes, lo que evita la contaminación espacial entre tramos diferentes de la pista y permite evaluar las propiedades morfológicas de cada figura individualmente.

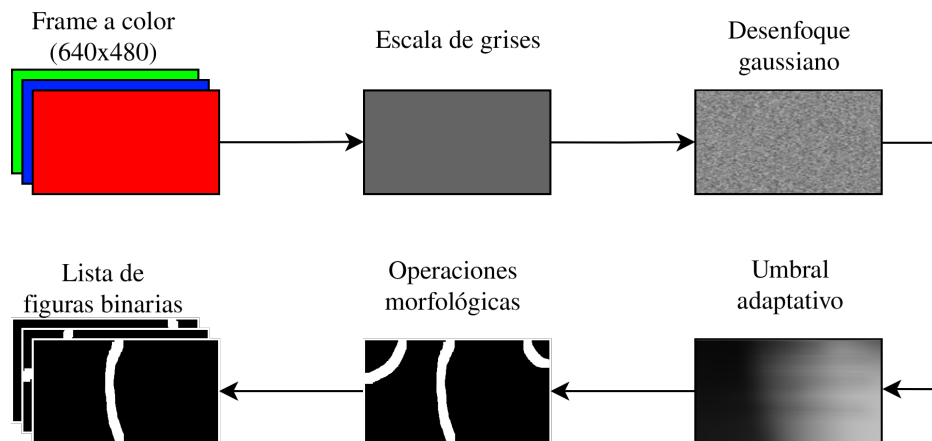


Figura 5.4: Pipeline de binarización adaptativa de un frame a color

5.5. Análisis geométrico de la trayectoria

Con la lista de figuras binarias independientes correctamente aisladas, el sistema debe extraer sus propiedades espaciales para modelar matemáticamente la ruta y permitir el cálculo de los errores que alimentarán a los controladores PID de vuelo.

5.5.1. Cálculo del índice de solidez

El descriptor geométrico fundamental utilizado para discriminar la naturaleza morfológica del camino es el Índice de Solidez. Este índice es una medida adimensional que evalúa la compacidad de un contorno comparando su área real frente al área de su casco convexo (*Convex Hull*). El casco convexo representa matemáticamente la envoltura convexa mínima que encierra al contorno, calculada en OpenCV mediante el algoritmo de Sklansky [23]. La fórmula matemática para la extracción de este índice es:

$$\text{Solidez} = \frac{\text{Área de la Figura}}{\text{Área del Casco Convexo}} \quad (5.2)$$

La utilidad de este parámetro en el proyecto es elevada, como veremos en el cálculo dinámico de la velocidad (5.6). De esta manera, podremos diferenciar matemáticamente distintos tipos de tramos:

- **Tramos rectos:** Una línea recta perfecta presenta un contorno cuyos límites coinciden casi en su totalidad con su casco convexo, por lo que el valor del índice de solidez es muy elevado, aproximándose de forma asintótica a 1.0.
- **Tramos curvos o bifurcaciones:** Cuando la trayectoria experimenta una curvatura o se divide en una bifurcación, la envoltura del casco convexo abarca grandes regiones de espacio vacío que no contienen píxeles negros reales (la concavidad interna de la curva). Esto provoca que el área del casco convexo crezca de manera desproporcionada en comparación con el área real de la línea, haciendo que el índice de solidez caiga significativamente hacia valores inferiores.

Este comportamiento matemático permite al software categorizar el tipo de tramo que sobrevuela el dron en tiempo real, sirviendo como un indicador predictivo para regular los parámetros de velocidad.



(a) Imagen original



(b) Imagen binarizada



(c) Casco convexo

5.5.2. Técnica de segmentación por ventanas de corte

Para determinar los puntos de paso de la trayectoria a lo largo de los márgenes del fotograma, se analiza la distribución de píxeles binarios en regiones estrechas de 10 píxeles de ancho, localizadas en los cuatro extremos del frame (superior, inferior, izquierdo y derecho). Adicionalmente, se establece un corte horizontal de referencia posicional situado exactamente en la fila $y = H - 133$. Esta coordenada representa geométricamente la proyección virtual del centro de masas del dron sobre el plano del suelo.

Para suprimir lecturas erróneas causadas por el efecto de borde en las esquinas de la imagen, las celdas extremas de los vectores de corte se fuerzan a 0. Posteriormente, se aplica el operador de diferenciación discreta *numpy.diff* [24] sobre las matrices de corte. Este operador actúa como un detector de bordes unidimensional que identifica las transiciones de color de blanco a negro:

$$\Delta I = I[n + 1] - I[n] \quad (5.3)$$

Para garantizar la consistencia espacial de la línea detectada frente al ruido residual, el algoritmo implementa un filtro basado en estadísticas de votación. Agrupa los puntos de transición detectados y evalúa la longitud de los pares de intersección. Únicamente se validan y calculan los puntos medios de aquellas secciones cuya anchura corresponda a la longitud de par más común de la matriz de corte, descartando de esta manera interferencias del entorno. Los cortes resultantes se estructuran en una lista

de coordenadas en píxeles que representan las salidas de la línea por el contorno del fotograma: arriba, abajo, izquierda, derecha y proyección del dron.

5.5.3. Selección de trayectoria

Se realiza un prefiltrado geométrico convirtiendo las coordenadas a un espacio angular desde el centro de la imagen mediante $\arctan2$. Aquellos cortes que se encuentren a una distancia angular inferior a un umbral crítico de 5.0 grados se consolidan como un único elemento centrado, evitando dobles consideraciones en puntos críticos como las esquinas de un frame o pequeñas imperfecciones en un mismo corte.

El mecanismo de decisión encargado de fijar el rumbo del dron recibe el conjunto de figuras binarias con sus respectivos cortes y aplica un criterio de persistencia de cercanía mínima para evitar que el dron sufra pérdidas:

1. **Búsqueda del contorno objetivo:** Evalúa de manera iterativa todos los cortes en los límites superior, izquierdo y derecho de cada figura disponible. Calcula la distancia euclidiana espacial entre cada corte candidato y el último punto de guiado válido registrado en el fotograma anterior. La figura cuyo corte minimiza esta distancia es seleccionada como el camino correcto a seguir, garantizando la continuidad del vuelo.
2. **Resolución de bifurcaciones por emparejamiento central:** Una vez fijada la figura, asociamos cada corte de los límites perimetrales externos con el punto de corte correspondiente de la horizontal de proyección del dron sobre la línea negra. El emparejamiento se realiza buscando el mínimo de la distancia euclidiana entre las combinaciones de puntos. Si existen múltiples parejas válidas (escenario característico de una bifurcación), el algoritmo evalúa la preferencia de bifurcación gobernada por las señales LED del entorno: si la preferencia es '*izquierda*', selecciona la pareja asociada al extremo izquierdo; si es '*derecha*', opta por la de la derecha.
3. **Cálculo del ángulo de guiñada:** Con la pareja de puntos definitiva seleccionada, compuesta por el punto de corte de la proyección (x_c, y_c) y el punto del borde exterior (x_e, y_e) , se calcula el ángulo de orientación matemática a corregir:

$$\begin{aligned} \Delta x &= x_e - x_c \quad , \quad \Delta y = y_c - y_e \\ \theta_{\text{yaw}} &= -\arctan2(\Delta y, \Delta x) \cdot \left(\frac{180}{\pi} \right) \end{aligned} \tag{5.4}$$

Este valor angular en grados se introduce de forma directa como el error del proceso en el controlador PID de orientación, el cual regula la velocidad angular de rotación sobre el eje vertical del dron. El punto del borde seleccionado se almacena como el nuevo último punto de guiado.

5.5.4. Cálculo del desplazamiento lateral

Para corregir las derivas horizontales y asegurar que el centro de masas del dron coincida exactamente con la vertical del eje de la cinta adhesiva, se calcula la desviación en píxeles. Esto mide la distancia horizontal desde la coordenada en el eje X del corte horizontal hasta el centro de la imagen proyectada, fijado a la mitad del ancho de resolución (320 píxeles para un fotograma de resolución 640x480 píxeles).

Para transformar este error medido en el dominio de la imagen, medido en píxeles, al dominio físico real, expresado en centímetros, se implementa una formulación que considera la altura instantánea de la aeronave proporcionada por el sensor de tiempo de vuelo (*ToF*) integrado en el hardware y el campo de visión (*FOV*) diagonal de la óptica de la cámara, el cual se sitúa en 82.6 grados. Dividiendo este ángulo en componentes simétricas respecto al eje óptico (41.3 grados), el error lateral métrico final se obtiene mediante la siguiente relación:

$$Roll_{\text{offset}} = \frac{Altura_ToF \cdot \tan(41,3^\circ) \cdot (X_c - 320)}{320} \quad (5.5)$$

5.6. Modulación dinámica de la velocidad

Una de las características del sistema visual implementado es su capacidad para regular de manera autónoma la velocidad de avance longitudinal (*Pitch*), adaptándose de forma dinámica a la complejidad del trazado.

La rutina evalúa simultáneamente tres parámetros clave extraídos por las etapas de visión previas. A partir de estos tres componentes, el algoritmo calcula perfiles de velocidad independientes utilizando una interpolación lineal por tramos acotada entre límites estrictos de operación segura, con una velocidad mínima de 10 centímetros por segundo y una máxima de 40:

- **Perfil basado en Solidez** (v_{sol}): Si la solidez del contorno es alta ($\geq 0,8$), se interpreta que el dron transita por una recta despejada, asignando la velocidad máxima de 40 cm/s. Si la solidez cae hacia 0.0 debido a la concavidad de un giro o una bifurcación, la velocidad se reduce de forma preventiva hacia el mínimo de 10 cm/s:

$$v_{\text{sol}} = \text{interp}(\text{Solidez}, [0,8, 0,0], [40, 10])$$

- **Perfil basado en Yaw** (v_{yaw}): Si el controlador de rumbo detecta que se requiere una corrección angular severa (valores absolutos de giro próximos o superiores a 10 grados/s), la velocidad de avance se reduce de forma drástica hacia el mínimo para evitar que el dron describa una trayectoria tangente y se salga de la pista:

$$v_{\text{yaw}} = \text{interp}(|\theta_{\text{yaw}}|, [0, 10], [40, 10])$$

- **Perfil basado en Roll** (v_{rol}): De igual manera, si el desplazamiento lateral respecto al eje es elevado (corrección de roll próxima a los 20 cm), el avance se

reduce para priorizar la estabilización y recentrado del vehículo:

$$v_{\text{rol}} = \text{interp}(|\text{Roll}_{\text{out}}|, [0, 20], [40, 10])$$

El comando de velocidad lineal definitivo enviado al dron se obtiene mediante el cálculo de la media aritmética ponderada de los parámetros individuales, duplicando el peso del factor de rumbo debido a su criticidad en la estabilidad del lazo:

$$\text{velocidad} = \text{mean}([v_{\text{sol}}, v_{\text{yaw}}, v_{\text{yaw}}, v_{\text{rol}}])$$

Esto permite una navegación más acelerada en rectas y una desaceleración en curvas pronunciadas y en acercamientos a bifurcaciones.

5.7. Integración de marcadores ArUco

Como complemento al sistema de guiado continuo por línea y control semántico por LEDs, el pipeline de visión incluye la infraestructura necesaria para la detección de marcadores ArUco.

El sistema utiliza funciones de OpenCV y diccionarios optimizados para aplicaciones de robótica aérea debido a su bajo número de bits, lo que reduce la carga computacional de la ventana de búsqueda y minimiza la tasa de falsos positivos. El algoritmo opera convirtiendo el fotograma reescalado a escala de grises y ejecutando el detector especializado con parámetros de detección estándar. Al localizar un marcador válido, el módulo extrae su identificador numérico único (ID) y las coordenadas de sus cuatro esquinas en el plano de la imagen (esquinas).

Aunque la arquitectura del módulo tiene como finalidad última servir de base para un mapa topológico, su utilidad actual responde directamente a necesidades de seguridad ante contingencias en la planta mediante la codificación de acciones a través de los identificadores únicos de los marcadores. De este modo, la lógica de control está configurada para que, al identificar un marcador en la pista, el dron realice la acción codificada en ese identificador (parada o aterrizaje). Esto permite detener el vehículo aéreo de forma autónoma o cortar el camino antes de que se adentre en un tramo afectado por averías técnicas o por la presencia de maquinaria pesada.

6. Sistema de Control y Navegación

Este capítulo detalla el diseño, la lógica de ejecución y la arquitectura de control en lazo cerrado que controla el comportamiento dinámico de la aeronave. Una vez que el sistema de visión por computador (desarrollado en el capítulo anterior) procesa los fotogramas y entrega las variables de error geométricas y de estado, estas actúan como entradas fijas para este subsistema. A partir de esta información, la arquitectura coordina las fases operativas del vuelo a través de una máquina de estados y ejecuta las correcciones cinemáticas necesarias para estabilizar la aeronave y guiarla con precisión a lo largo del circuito.

6.1. Máquina de estados y toma de decisiones

El objetivo central es que el dron navegue de forma autónoma siguiendo un camino predefinido y tome decisiones operativas basadas exclusivamente en la interpretación visual de señales cromáticas en pista.

El flujo de operaciones principal se estructura en las siguientes etapas secuenciales después de establecerse correctamente la conexión inicial con el dron:

1. **Despegue y estabilización:** El dron ejecuta una secuencia de despegue seguro hasta alcanzar una altitud de consigna establecida en 60 centímetros respecto al suelo (ajustable mediante argumentos en la ejecución del programa).
2. **Calibración lumínica opcional:** Si se requiere, se activa el protocolo de calibración de los diodos LED. El sistema muestra en la interfaz el color que debe configurar el operador en la tira. Tras una pausa de 3 segundos para el cambio físico de color, se inicia una ventana de muestreo de 3 segundos. En esta fase, se registran los valores de matiz (Hue) de los píxeles que superan los umbrales de intensidad ($V > 190$ y $S > 30$) para modelar las curvas gaussianas (media μ y desviación estándar σ) que definirán las fronteras de decisión dinámicas.
3. **Bucle de navegación principal:** Una vez iniciado, el sistema solicita al búfer de vídeo del dron. Se aplican mecanismos defensivos de control de flujo, explicados en el capítulo anterior.

Durante la navegación, las decisiones de alto nivel de la máquina de estados están gobernadas por la paleta cromática previamente definida (5.2.4). Para garantizar la robustez del sistema ante la conmutación física de los diodos, las órdenes se seleccionan calculando la moda estadística de la cola FIFO de 5 elementos.

Esta lógica de eventos se coordina en paralelo con el subsistema de integración de marcadores ArUco (5.7). Dentro de la arquitectura de control, los marcadores actúan con una prioridad jerárquica superior a las señales LED. Si el sistema detecta una baliza con un ID válido, se anula cualquier comando proveniente de la tira LED, reanudando su normal funcionamiento cuando este marcador se elimina del rango visual del dron.

6.2. Arquitectura controladores PID

Debido a las perturbaciones aerodinámicas, como el efecto suelo, que sufre un UAV de 80 gramos de peso en espacios pequeños, es necesario que el control de la trayectoria se gestione mediante una arquitectura de doble lazo cerrado que opere en paralelo. La implementación sufrió una serie de cambios a lo largo del proyecto, mitigando los errores percibidos en fases anteriores.

6.2.1. Evolución de la arquitectura del sistema

En las fases iniciales del proyecto, el sistema disponía de un único controlador Proporcional-Integral-Derivativo (PID) asignado en exclusiva al eje de orientación angular (*Yaw*). La hipótesis de diseño inicial asumía que orientando la proa del dron de forma paralela a la línea negra, el vehículo completaría el circuito de forma natural. Sin embargo, las pruebas desvelaron una inestabilidad estructural condicionada por la naturaleza física del entorno: el circuito está confeccionado sobre una lona blanca lisa con una textura homogénea.

Esta uniformidad visual del fondo provocaba que el sistema de posicionamiento visual nativo del dron (VPS) fuera incapaz de estimar correctamente la velocidad de deriva horizontal de la plataforma. Como resultado, aunque el lazo de guiñada mantenía al dron perfectamente alineado en rumbo con la cinta, la aeronave se desplazaba lateralmente de forma incontrolada hasta salirse por los márgenes de la pista.

Para mitigar este fenómeno y estabilizar la navegación, se implementó una estrategia mixta, tanto a nivel de hardware como de software:

- **Modificación del Entorno (*Hardware*):** Se añadieron de pegatinas de formas irregulares de alto contraste en los laterales de las rectas del circuito. Estos elementos funcionan como puntos de anclaje visual artificial, añadiendo los gradientes ópticos necesarios para que el sensor *VPS* inferior estabilice de forma autónoma la odometría horizontal del dron.
- **Estructura de Doble Lazo PD (*Software*):** Se incorporó un segundo controlador independiente para el eje de alabeo (*Roll*). Bajo este esquema dual, ambos reguladores operan de forma simultánea en cada iteración: mientras el lazo de *Yaw* corrige el rumbo angular del vehículo, el lazo de *Roll* procesa el desplazamiento lateral para contrarrestar la deriva lateral inercial y mover el centro de masas del dron exactamente sobre la vertical de la cinta negra.

6.2.2. Sintonización de las constantes

Debido a que el entorno de pruebas se encuentra en una zona interior donde interferencias externas como el viento no están presentes, se determinó anular por completo la componente integral en ambos controladores ($K_i = 0,0$), transformando la topología del sistema en un doble lazo Proporcional-Derivativo (PD).

El ajuste de las ganancias numéricas se llevó a cabo mediante el método heurístico de tanteo sistemático sobre el circuito tras la estabilización del VPS con las pegatinas de anclaje. Las constantes definitivas integradas de manera estable en el código fuente de la aplicación se detallan a continuación:

- **Controlador PD de Orientación (Yaw):**

$$K_p = 1,5 \quad , \quad K_i = 0,0 \quad , \quad K_d = 0,1$$

La salida de este controlador se limita a un rango estricto de $[-45, 45]^\circ/\text{s}$. Esta restricción impide que el dron demande velocidades de rotación excesivas que emborronarían la captura de vídeo (*motion blur*) y degradarían la segmentación.

- **Controlador PD de Desplazamiento Lateral (Roll):**

$$K_p = 1,15 \quad , \quad K_i = 0,0 \quad , \quad K_d = 0,55$$

La acción de control se acota en el rango de $[-20, 20]$ cm/s para asegurar traslaciones laterales suaves, amortiguando la inercia sin producir respuestas bruscas que expulsan al UAV del camino textil.

6.3. Envío de comandos de vuelo

La última etapa del bucle de navegación corresponde al bloque de actuación, encargado de traducir las salidas continuas de los controladores cinemáticos y los perfiles de velocidad longitudinal en comandos discretos directamente interpretables por la *API* del DJI Tello.

La interfaz de programación del dron requiere la recepción de tramas de control estructuradas en cuatro canales de velocidad independientes, normalizados obligatoriamente en un rango de valores enteros comprendido entre $[-100, 100]$. La transferencia de las variables se realiza bajo los siguientes criterios:

- **Canal de Alabeo (Roll):** Está gobernado directamente por la acción de control de velocidad lateral calculada por el PID de Alabeo (dentro del rango limitado a ± 20). Regula el desplazamiento a la derecha (valores positivos) o a la izquierda (valores negativos).
- **Canal de Cabeceo (Pitch):** Está gobernado por la velocidad de avance dinámica calculada mediante la interpolación lineal (5.6). Siendo que comprenderá el rango de velocidades $[10, 40]$ cm/s en todo momento.
- **Canal de Guiñada (Yaw):** Está gobernado por la salida del PID de Guiñada (dentro del rango ± 45). Controla la velocidad de rotación angular sobre el eje vertical en sentido horario (valores positivos) o antihorario (valores negativos).
- **Canal de Aceleración (Throttle):** Se mantiene de forma fija en un valor de 0. Al fijar esta velocidad en cero, se delega el control de la altitud y la compensación del efecto suelo a los bucles de regulación automática del hardware del Tello. Esto permite que el dron mantenga de forma autónoma la altura establecida en la etapa de despegue seguro inicial sin requerir cómputo externo adicional.

6.4. Modularidad y portabilidad del hardware

Si bien esta configuración de red inalámbrica ha permitido validar de forma experimental la robustez del pipeline visual y la precisión del control PID en un entorno controlado de laboratorio, la dependencia de un canal inalámbrico para el intercambio de datos en lazo cerrado no satisface los requisitos de robustez ni las restricciones de seguridad física exigidas en los entornos industriales descritos en la introducción. Por este motivo, la arquitectura de software desarrollada en este proyecto se ha diseñado con un alto nivel de modularidad y desacoplamiento.

El objetivo final es portar la totalidad del pipeline dentro de un ordenador monoplaca de bajo peso y reducidas dimensiones (estilo *Raspberry Pi*) montado físicamente a bordo de la propia estructura del dron. Esta evolución permitiría interconectar de forma directa la unidad de cómputo y la placa controladora de vuelo mediante una interfaz física cableada de comunicación serie, eliminando por completo la necesidad de conectividad inalámbrica durante la operación y logrando un sistema completamente autónomo e inmune a inhibiciones de señal o interferencias electromagnéticas. Lamentablemente, el dron DJI Tello solo permite conexión mediante Wi-Fi 2.4GHz, siendo que el puerto Micro USB solamente está disponible para la carga de la batería ¹, imposibilitando esta fase de integración de hardware externo.

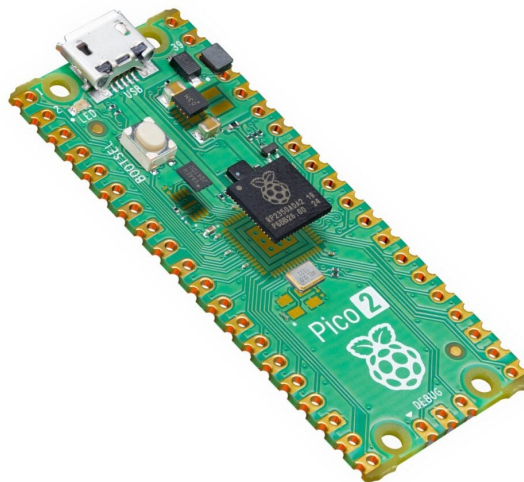


Figura 6.1: Ordenador monoplaca Raspberry Pi Pico 2 ²

¹DJI Tello Especificaciones - <https://www.ryzerobotics.com/es/tello/specs>

²Raspberry Pi Pico 2 - <https://www.raspberrypi.com/products/raspberry-pi-pico-2/>

7. Pruebas y Resultados

Este capítulo muestra las pruebas y los resultados obtenidos tras el despliegue del sistema de navegación autónoma en el entorno de pruebas. Para evaluar de manera rigurosa el comportamiento global del sistema, las pruebas se han dividido en: analizar la eficiencia temporal y la robustez algorítmica del pipeline de visión artificial y evaluar la estabilidad cinemática de los lazos cerrados de control y la repercusión de las optimizaciones físicas introducidas en el entorno de pruebas.

7.1. Rendimiento del sistema de visión por computador

El correcto funcionamiento del lazo de control depende de la precisión y velocidad con la que el módulo de percepción procesa los fotogramas. Las pruebas en este ámbito se orientaron a garantizar el determinismo y la inmunidad del sistema ante perturbaciones ópticas externas.

7.1.1. Telemetría y depuración gráfica en tiempo real

Para verificar el correcto desempeño del procesado de imagen sin penalizar el rendimiento en vuelo, se desarrolló una capa de telemetría mediante el argumento `--save-video` al ejecutar el programa. Esta rutina almacena de manera secuencial los fotogramas originales capturados por la cámara del DJI Tello, añadiendo sobre la matriz de píxeles una máscara gráfica con las variables de estado calculados en cada iteración.

Esta técnica permitió auditar visualmente y fotograma a fotograma los siguientes aspectos del pipeline:

- Los puntos de corte geométricos empleados para calcular el error angular (θ_{yaw}) y la desviación lateral en centímetros ($Roll_{offset}$).
- Los identificadores numéricos (ID) de los marcadores ArUco detectados y las cajas de delimitación de sus esquinas.
- El color resultante de la cola FIFO como resultado de analizar la tira de diodos LED.

El análisis de estas grabaciones confirmó la corrección del pipeline de visión en múltiples pruebas realizadas en todo tipo de escenarios distintos. Siendo que esta rutina de telemetría no supone un riesgo temporal, puesto que entra dentro del margen de tiempo estricto de 33 milisegundos de cómputo entre fotogramas, como veremos en subsecciones posteriores.

7.1.2. Robustez del sistema lumínico mediante calibración

Las primeras aproximaciones del proyecto empleaban umbrales fijos dentro del espacio de color HSV para segmentar las órdenes de la tira LED. Sin embargo, los experimentos demostraron que este enfoque era sumamente vulnerable a factores externos, como la luminosidad variable del entorno de pruebas a distintas horas del día, así como factores físicos de la tira LED como diferencia de brillo entre colores (azul más brillante que naranja).

La introducción del parámetro `--calibrate-led` solucionó esta problemática al sustituir los límites estáticos por un modelo de calibración estadística gaussiana previa al vuelo. Durante esta fase, el software muestrea el entorno real de la pista y calcula la media (μ) y la desviación estándar (σ) del canal H para cada color. Al desplazar la frontera de decisión de una definición teórica ideal hacia una definición práctica adaptada al entorno, la tasa de falsos positivos en la clasificación cromática se redujo a valores despreciables, garantizando la estabilidad de la máquina de estados.

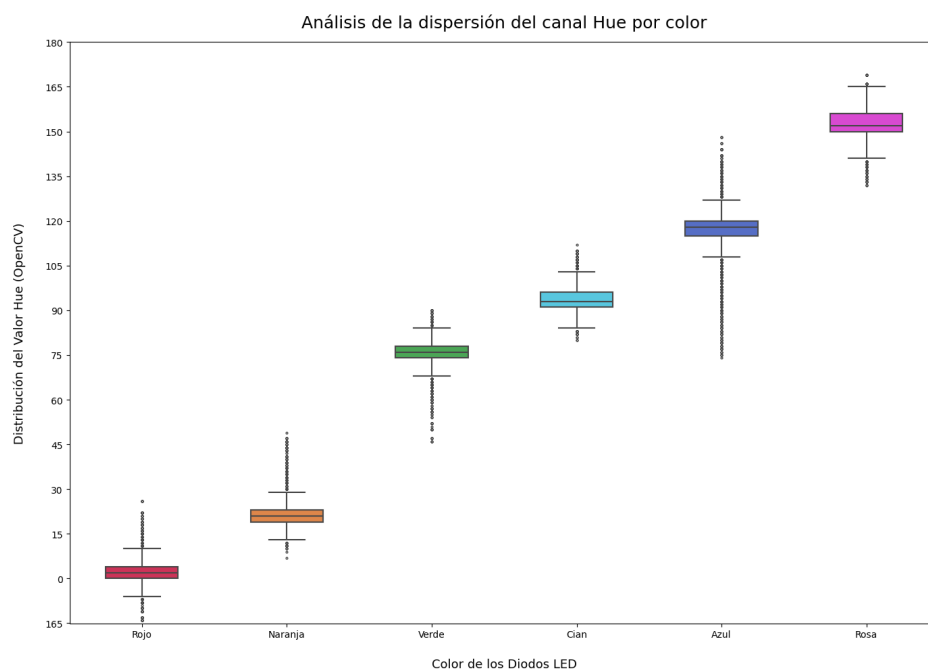


Figura 7.1: Distribución de los valores Hue de cada color en una calibración

Si bien esta calibración redujo de manera considerable la tasa de falsos positivos seguía existiendo el problema de la transición entre colores. Esto pudo solventarse mediante la implementación de una cola FIFO de 5 elementos (5.2.3).

7.1.3. Análisis temporal y de concurrencia

El flujo de vídeo del DJI Tello opera a una tasa de 30 fotogramas por segundo (FPS), lo que establece un presupuesto de tiempo crítico de 33,33 ms por fotograma.

Si el tiempo de cómputo total de una iteración supera este umbral, se produce una acumulación de frames en el búfer de red, provocando retrasos acumulativos que podrían desestabilizar el control físico del dron.

Para evaluar este aspecto, se integró un módulo de monitorización temporal mediante el argumento `--timer` que registra la latencia interna de cada subrutina en milisegundos. Como se ilustra en el diagrama de tiempos de la Figura 7.2, el pipeline se ejecuta con un margen de seguridad elevado respecto al cuello de botella del sistema.

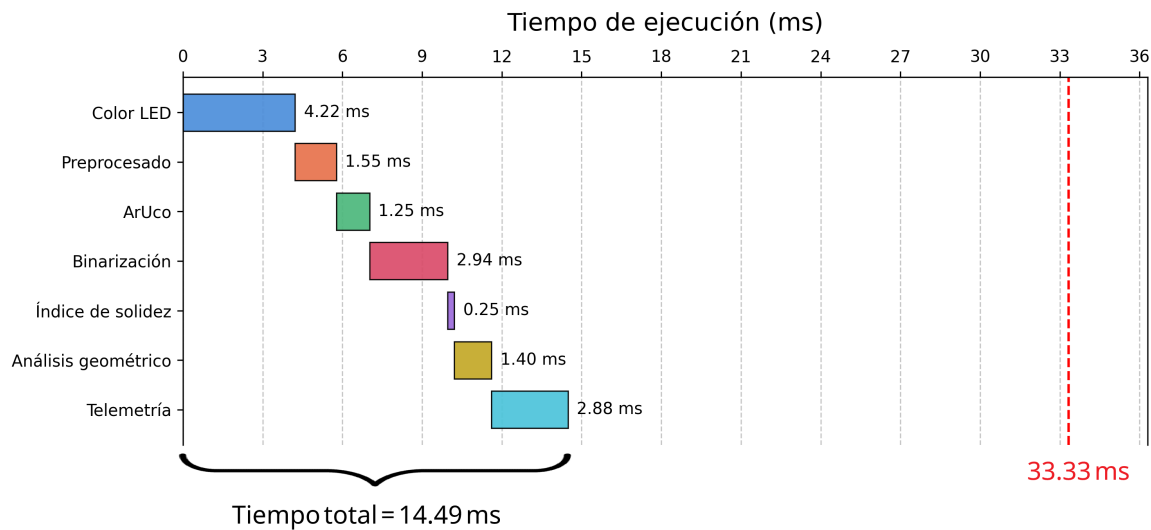


Figura 7.2: Diagrama Gantt del tiempo medio de ejecución de cada tarea del pipeline de visión.

Asimismo, la distribución de tiempos de ejecución entre fotogramas (véase la Figura 7.3) no supera en ningún caso los 33,33 ms. (tiempo entre cada fotograma del vídeo a 30 FPS).

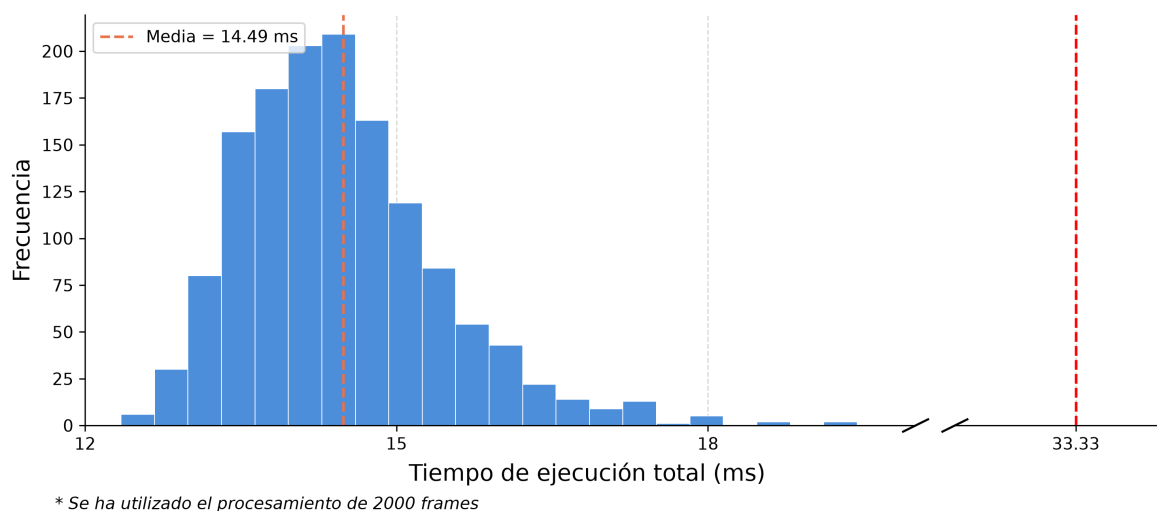


Figura 7.3: Histograma de la distribución del tiempo de ejecución entre fotogramas.

Este rendimiento se fundamenta en dos optimizaciones clave:

1. **Filtro de descarte de fotogramas duplicados:** Antes de iniciar el procesamiento morfológico, el algoritmo compara la matriz del fotograma actual con la del anterior. Si son idénticas, detiene la ejecución durante 1 milisegundo y espera el siguiente refresco de la cámara, ahorrando ciclos de CPU críticos en la estación de tierra.
2. **Reducción de escala métrica:** El redimensionamiento controlado de la imagen de entrada disminuye drásticamente el número de píxeles a evaluar en las funciones de detección de bordes (`get_edge_cuts`), binarización (`binarize_frame`) y detección de marcadores ArUco (`aruco_markers`) que son las tareas mas demandantes del pipeline.

7.2. Evolución del sistema de guiado

La estrategia de guiado del cuadricóptero experimentó una evolución a lo largo del proyecto, condicionada por los resultados obtenidos en las sucesivas fases de pruebas en pista. Asimismo, la velocidad del dron también es algo que sufrió cambios conforme se modificó esta estrategia de guiado.

7.2.1. Desarrollo del bucle de control

El desarrollo de la arquitectura de navegación se dividió en cuatro etapas:

- **Fase 1: Control en lazo abierto y velocidad constante.** Inicialmente, el dron operaba introduciendo el error de orientación (*Yaw*) sin controladores PID y aplicando velocidades fijas el canal de *Pitch*, asumiendo que la trayectoria se mantendría recta de forma natural. Los resultados fueron deficientes debido a que cualquier perturbación aerodinámica desviaba al dron del camino.
- **Fase 2: Implementación del lazo cerrado de orientación (*Yaw*).** Se incorporó un controlador Proporcional (P) al error angular de la línea. Aunque el dron lograba corregir su rumbo, el sistema sufría de una fuerte oscilación en el eje vertical debido a la inercia del UAV. La adición del término Derivativo (D) mitigó este efecto, amortiguando la respuesta y estabilizando el rumbo del dron en las rectas.
- **Fase 3: Expansión a arquitectura de doble lazo cerrado (*Yaw* + *Roll*).** Una vez estabilizada la orientación, las pruebas en pista desvelaron un desfase cinemático: el dron avanzaba manteniendo su proa perfectamente paralela a la trayectoria, pero sufría un desplazamiento lateral acumulativo, volando de lado y saliéndose lateralmente del circuito. Para solucionar este desacoplamiento de ejes, se diseñó e implementó en paralelo un segundo lazo cerrado Proporcional-Derivativo (PD) asignado al canal de alabeo (*Roll*). Este regulador comenzó a procesar activamente el desplazamiento lateral en centímetros respecto al cen-

troide de la línea, aplicando comandos de empuje opuestos para fijar de manera definitiva el centro de masas del dron sobre la vertical del camino.

- **Optimización final: Estabilización del VPS mediante puntos de anclaje.** Con ambos lazos cerrados operando simultáneamente (Yaw + Roll), el sistema alcanzó una alta precisión de guiado; sin embargo, surgieron inestabilidades ocasionales. La tela blanca carecía de texturas, lo que confundía al Sistema de Posicionamiento por Visión (*VPS*) del DJI Tello, impidiéndole estimar correctamente su propia velocidad. El hito final que consiguió la estabilidad total del vuelo fue la distribución estratégica de pegatinas negras o puntos de anclaje a los lados del camino principal. Estas referencias ópticas artificiales proporcionaron al *VPS* las texturas heterogéneas visuales necesarias para bloquear el posicionamiento inercial del dron, eliminando por completo las pérdidas accidentales de estabilidad.

Tras un proceso de sintonización heurística en pista, las funciones de transferencia se consolidaron con los siguientes parámetros definitivos:

$$\text{Regulador Yaw: } K_p = 1,5, \quad K_i = 0,0, \quad K_d = 0,1 \quad (\text{Saturación: } \pm 45^\circ/\text{s}) \quad (7.1)$$

$$\text{Regulador Roll: } K_p = 1,15, \quad K_i = 0,0, \quad K_d = 0,55 \quad (\text{Saturación: } \pm 20 \text{ cm/s}) \quad (7.2)$$

7.2.2. Justificación de técnica de sintonización del controlador

Al existir dos lazos de control dependientes entre sí, el comportamiento global está acoplado, lo que dificulta la aplicación directa de métodos clásicos. De esta manera, la ausencia de un modelo matemático preciso del entorno de pruebas, unida a la no linealidad del dron, imposibilita el uso de los métodos analíticos.

Asimismo, técnicas como Ziegler-Nichols presentan algunas restricciones, ya que al realizarse la sintonización de las componentes con el dron en un entorno físico existe la posibilidad de que el sistema se salga de control debido a la inestabilidad del controlador, ocasionando daños físicos al dron.

Por tanto, el ajuste heurístico secuencial demostró ser el método más seguro y eficaz, procediendo de la manera expuesta en el subapartado anterior (7.2.1).

7.2.3. Perfil de velocidad dinámica

A diferencia de las primeras versiones de velocidad estática, la versión final implementa la función `get_speed` que calcula el empuje frontal (*Pitch*) de manera adaptativa. El sistema monitoriza de forma continua tres variables de estado: la solidez del camino segmentado (que cae drásticamente ante la proximidad de curvas o bifurcaciones) y los valores absolutos de las salidas de los controladores de *Yaw* y *Roll*.

Mediante una interpolación lineal acotada entre los rangos de $[10, 40]$ cm/s, el dron reduce de forma autónoma su avance al aproximarse a tramos complejos o cuando acumula un error de centrado elevado. Esto otorga una prioridad a la maniobra de

reorientación y recentrado sobre la velocidad de aceleración, reduciendo las salidas de pista a cero en la configuración final del circuito.

7.3. Análisis de la evolución global del proyecto

El éxito final del prototipo autónomo es el resultado directo de la interconexión de optimizaciones sobre el hardware, el software y el propio entorno físico. La Tabla 7.1 sintetiza el impacto técnico de las cuatro decisiones más relevantes adoptadas durante el desarrollo del proyecto.

Tabla 7.1

Optimizaciones globales incorporadas al sistema.

Modificación implementada	Problema detectado en pruebas	Efecto conseguido
Rediseño del soporte del espejo en Blender ($45^\circ \rightarrow 50^\circ$).	Campo de visión excesivamente cenital y falta de anticipación ante curvas cerradas.	Incremento de la anticipación, con una mayor ventana de reacción.
Transición de un único lazo de control a arquitectura doble.	Desviación lateral acumulativa provocada por la inercia del cuadricóptero.	Prioridad de estabilización simultánea de rumbo (<i>Yaw</i>) y centrado (<i>Roll</i>) frente a la velocidad de desplazamiento.
Adición de pegatinas en los laterales de la línea del circuito de pruebas.	Pérdida de odometría del sensor <i>VPS</i> sobre fondo blanco uniforme.	Generación de patrones heterogéneos artificiales y estabilización del posicionamiento del dron.
Optimización por descarte de fotogramas idénticos secuenciales.	Consumo innecesario de ciclos de CPU en la estación de tierra o servidor.	Mitigación del efecto de cuello de botella y latencia media de procesamiento mantenida bajo los 33 ms.

Parte III

Epílogo

8. Conclusiones y Trabajo Futuro

Este capítulo sintetiza los resultados obtenidos y los conocimientos consolidados a lo largo del diseño, implementación y validación experimental del sistema de guiado autónomo. En primer lugar, se detallan las lecciones aprendidas, evaluando cómo las decisiones físicas y algorítmicas condicionaron el desempeño del vehículo aéreo. En segundo lugar, se definen las líneas de trabajo futuro orientadas a evolucionar el prototipo actual hacia una solución de grado industrial.

8.1. Lecciones aprendidas

El desarrollo e integración de un pipeline de visión artificial y control en lazo cerrado sobre una plataforma con severas restricciones de cómputo y conectividad ha proporcionado valiosas conclusiones técnicas:

1. **Dependencia crítica entre hardware y software:** Las pruebas experimentales demostraron que las limitaciones de un algoritmo de percepción visual no siempre se resuelven optimizando el código, sino modificando la física del sensor. El soporte original del espejo impreso en 3D con una inclinación de 45° limitaba la visión del dron a un plano estrictamente cenital. Al rediseñar la geometría de la pieza en Blender a un ángulo de 50° , se duplicó la ventana visual frontal (pasando de 20 a 40 centímetros de pista visible por delante del dron). Este incremento en la anticipación visual proporcionó al software el tiempo de reacción necesario para procesar tramos complejos y ejecutar desaceleraciones dinámicas efectivas antes de la entrada en curvas o bifurcaciones.
2. **Complejidad cinemática en entornos de baja textura:** El diseño de la arquitectura de guiado evidenció los desafíos asociados al acoplamiento de variables dinámicas independientes en drones. El despliegue inicial de un único lazo cerrado enfocado en el eje de orientación (*Yaw*) resultó insuficiente debido al comportamiento inercial del vehículo. La superficie uniforme de la lona blanca anulaba la eficacia del sensor de flujo óptico interno (*VPS*) del dron, provocando desvíos laterales acumulativos. La solución requirió un enfoque dual: por un lado, la implementación de un segundo controlador Proporcional-Derivativo (PD) dedicado en exclusiva al canal de alabeo (*Roll*) para corregir la desviación transversal y, por otro lado, la adaptación física del entorno mediante pegatinas que actuaron como puntos de anclaje inerciales para estabilizar el vuelo.
3. **Modelado estadístico frente a determinismo rígido:** Se constató que las soluciones de software basadas en umbrales fijos son inviables en escenarios del mundo real debido a factores externos inevitables, como la caída de tensión en las baterías de los diodos LED o las oscilaciones en la iluminación del laboratorio. La implementación del protocolo de calibración adaptativa demostró que la robustez de un sistema de visión artificial no radica en intentar evitar las perturbaciones

ambientales, sino en modelarlas matemáticamente en tiempo real, reduciendo prácticamente a cero la tasa de falsos positivos en la clasificación cromática.

4. **Determinismo temporal como requisito de estabilidad:** En sistemas robóticos en lazo cerrado, la eficiencia algorítmica está directamente ligada a la seguridad física. El procesamiento de vídeo del DJI Tello impone una restricción en tiempo real estricta de 33,33 ms por fotograma. El desarrollo de técnicas de optimización específicas, como la reducción de escala de la matriz de imagen y el filtro de descarte de fotogramas idénticos duplicados, resultó crítico para garantizar que la carga computacional del pipeline de visión se mantuviera siempre por debajo de dicho umbral, evitando retrasos acumulativos que desestabilizaran los comandos enviados a los motores.

8.2. Trabajo futuro

A partir de la plataforma funcional obtenida, se identifican tres líneas de investigación y desarrollo para aproximar el prototipo a las necesidades operativas de una planta real:

1. **Navegación basada en mapas topológicos dinámicos y estaciones ArUco:** En el estado actual, el dron reacciona de manera aislada ante los marcadores ArUco, utilizándolos principalmente como mecanismos prioritarios. Se propone como evolución principal el diseño de un algoritmo que construya, en tiempo real y de forma dinámica, un mapa topológico estructurado como un grafo de nodos (intersecciones, bifurcaciones o estaciones) y aristas (tramos de línea negra). Bajo este paradigma, cada marcador ArUco se registrará con su identificador numérico único que representará una estación fija. Al integrar este mapa con la máquina de estados del clasificador cromático, el dron podrá procesar órdenes complejas de guiado; por ejemplo, ante una señal lumínica azul que indique la orden de retorno, el vehículo consultará de forma autónoma su mapa topológico guardado para planificar la ruta de navegación óptima y dirigirse de manera completamente automatizada a la base definida como su estación de origen (*HOME*), correspondiente a un ID específico predefinido para cada dron.
2. **Automatización del pipeline de visión mediante calibración avanzada:** A pesar de haber resuelto la adaptación frente a perturbaciones de luz en la tira LED, el pipeline de procesamiento visual aún mantiene ciertos umbrales fijos para las funciones de detección de focos luminosos. Como línea de mejora, se plantea sustituir de forma definitiva estos límites rígidos por técnicas de filtrado dinámico o, en su defecto, extender el módulo de calibración previa al vuelo. Este nuevo componente deberá muestrear analíticamente el contraste relativo de la pista antes de iniciar el despegue para deducir automáticamente los rangos de saturación y valor óptimos para la detección de señales luminosas bajo cualquier condición, asegurando que la carga computacional añadida no comprometa el presupuesto temporal crítico de los 33 ms del bucle principal.
3. **Migración a una arquitectura embebida:** Para satisfacer plenamente el requisito de diseño industrial propuesto originalmente, donde la dependencia de

redes inalámbricas representa una vulnerabilidad crítica ante la saturación de frecuencias o la presencia de interferencias electromagnéticas, es necesario eliminar el enlace Wi-Fi del bucle de control. Se propone como objetivo prioritario la portabilidad de todo el software algorítmico (pipeline de visión por computador, lógica de estados y reguladores PID) hacia un ordenador monoplaca de bajo peso y reducidas dimensiones (estilo *Raspberry Pi*) montado físicamente sobre el chasis de la aeronave. Esta unidad procesará el flujo de vídeo de la cámara a través de una conexión directa por cable a la placa controladora de vuelo del UAV, logrando un sistema de navegación en interiores completamente autónomo y con un nivel de seguridad óptimo para entornos de vulnerabilidad inalámbrica.

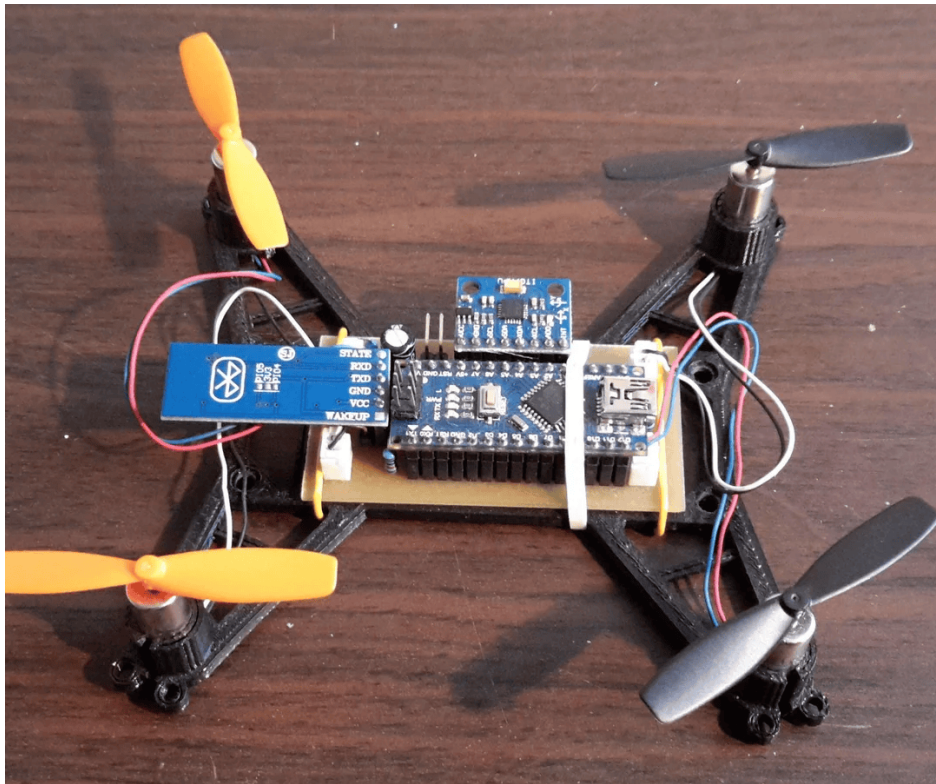


Figura 8.1: Dron con ordenador monoplaca embebido en la parte superior ¹.

¹Embedded Systems - <https://www.drishti.works/blog/tag/Embedded+Systems>

Información sobre Licencia

Todo el código fuente desarrollado en este proyecto se distribuye bajo la **Licencia MIT**. Se trata de una licencia de código abierto altamente permisiva que se caracteriza por:

- **Total libertad:** Permite a cualquier persona usar, modificar, copiar, fusionar, publicar, distribuir y vender el software sin restricciones.
- **Único requisito:** Obliga a mantener el aviso de *copyright* original del autor en cualquier copia o parte sustancial del código.
- **Exención de responsabilidad:** El software se proporciona "tal cual" (*as is*), lo que significa que el autor no ofrece garantías ni se hace responsable de posibles fallos o daños derivados de su uso.

Bibliografía

- [1] L. Muktilatin. DJI-Tello-LineFollowers. GitHub repository, 2026. [Online]. Available: <https://github.com/legsano/DJI-Tello-LineFollowers> [Accessed: Jun. 2026].
- [2] ukicomputers. Tello-AI. GitHub repository. [Online]. Available: <https://github.com/ukicomputers/Tello-AI> [Accessed: Jun. 2026].
- [3] D. Rathod. UAV-Line-Follower-Drone: Line Follower Algorithm for Parrot Mini Drone. GitHub repository. [Online]. Available: <https://github.com/drathod2/UAV-Line-Follower-Drone> [Accessed: Jun. 2026].
- [4] J. Martos Bianqui. Control automático con Simulink de un mini dron: Aplicación al seguimiento de trayectorias mediante realimentación visual. Trabajo fin de grado, Universitat Politècnica de València, Valencia, España, 2021. [Online]. Available: <https://riunet.upv.es/handle/10251/174861> [Accessed: Jun. 2026].
- [5] Koray Kızılay. Mathworks Minidrone Competition. GitHub repository. [Online]. Available: <https://github.com/koraykzly/parrotMinidroneCompetition> [Accessed: Jun. 2026].
- [6] MosaabDerbal. DJI-TELLO-Face-Tracking. GitHub repository. [Online]. Available: <https://github.com/mossaab3/DJI-TELLO-Face-Tracking> [Accessed: Jun. 2026].
- [7] Juan Manuel Pérez. Face Detection and Tracking with Dji Tello drone. GitHub repository. [Online]. Available: <https://github.com/juanmapf97/Tello-Face-Recognition> [Accessed: Jun. 2026].
- [8] O. García Ortiz. Sistema de detección, seguimiento y control PID para drones de interior y exterior con modos de juego. Trabajo fin de grado, Universitat Politècnica de Catalunya, Barcelona, España, 2026. [Online]. Available: <https://hdl.handle.net/2117/460067> [Accessed: Jun. 2026].
- [9] Nguyen Ngoc Luan. UAV line-following using image processing. GitHub repository. [Online]. Available: <https://github.com/NgocLuan/Line-flowing-DJI-TELLO-with-PD> [Accessed: Jun. 2026].
- [10] I. Hayik. Tello Arrow Navigator. Robotika Wiki. [Online]. Available: https://wiki.robotika.sk/robowiki/index.php?title=Tello_Arrow_Navigator_-_Ibrahim_Hayik [Accessed: Jun. 2026].
- [11] T. Szepessy. DJITelloOpticalControl. GitHub repository. [Online]. Available: <https://github.com/TamasSzepessy/DJITelloOpticalControl> [Accessed: Jun. 2026].

- [12] AlexJinlei. Autonomous UAVs Swarm Mission. GitHub repository. [Online]. Available: https://github.com/AlexJinlei/Autonomous_UAVs_Swarm_Mission [Accessed: Jun. 2026].
- [13] Ryze Robotics. *DJI Tello User Manual v1.0*, 2018. [Online]. Available: https://dl-cdn.ryzerobotics.com/downloads/Tello/201806mul/Tello%20User%20Manual%20V1.0_ES.pdf [Accessed: Jun. 2026].
- [14] Ryze Robotics. *DJI Tello SDK 2.0 Documentation v1.3*, 2018. [Online]. Available: <https://dl-cdn.ryzerobotics.com/downloads/Tello/Tello%20SDK%202.0%20User%20Guide.pdf> [Accessed: Jun. 2026].
- [15] OpenCV Development Team. Miscellaneous Image Transformations – adaptiveThreshold. OpenCV 4.x Documentation. [Online]. Available: https://docs.opencv.org/4.x/d7/d1b/group__imgproc__misc.html [Accessed: Jun. 2026].
- [16] S. Garrido-Jurado, R. Muñoz-Salinas, F. Madrid-Cuevas, and R. Medina-Carnicer. Generation of fiducial marker dictionaries using Mixed Integer Linear Programming. *Pattern Recognition*, 51, Oct. 2015.
- [17] Wikipedia contributors. Controlador PID. Wikipedia, La enciclopedia libre. [Online]. Available: https://es.wikipedia.org/wiki/Controlador_PID [Accessed: Jun. 2026].
- [18] Voltage Voyager. Understanding PI, PD, and PID Controllers: A Comprehensive Guide. Electrical Blog. [Online]. Available: <https://www.electrical-blog.com/understanding-pi-pd-and-pid-controllers-a-comprehensive-guide/#::~text=response%2E-,PD,critical> [Accessed: Jun. 2026].
- [19] A. Bhasin, J. Bennett, J. Grant, W. Chung Lim. Afinación PID a través de métodos clásicos. University of Michigan. [Online]. Available: [https://espanol.libretexts.org/Bookshelves/Ingenieria/Ingenieria_Industrial_y_de_Sistemas/Libro%3A_Din%C3%A1mica_y_Control_de_Procesos_Qu%C3%ADmicos_\(Woolf\)/09%3A_Control_proporcional-integral-derivado_\(PID\)/9.03%3A_Afinaci%C3%B3n_PID_a_trav%C3%A9s_de_m%C3%A9todos_cl%C3%A1sicos](https://espanol.libretexts.org/Bookshelves/Ingenieria/Ingenieria_Industrial_y_de_Sistemas/Libro%3A_Din%C3%A1mica_y_Control_de_Procesos_Qu%C3%ADmicos_(Woolf)/09%3A_Control_proporcional-integral-derivado_(PID)/9.03%3A_Afinaci%C3%B3n_PID_a_trav%C3%A9s_de_m%C3%A9todos_cl%C3%A1sicos) [Accessed: Jun. 2026].
- [20] T. Erez. DJI Tello Mirror Clip r.2. Thingiverse. [Online]. Available: <https://www.thingiverse.com/thing:4697733> [Accessed: Jun. 2026].
- [21] D. Fuentes. DJITelloPy. GitHub repository. [Online]. Available: <https://github.com/damiafuentes/DJITelloPy> [Accessed: Jun. 2026].
- [22] Wikipedia contributors. Desenfoque gaussiano. Wikipedia, La enciclopedia libre. [Online]. Available: https://es.wikipedia.org/wiki/Desenfoque_gaussiano [Accessed: Jun. 2026].
- [23] OpenCV Development Team. Structural Analysis and Shape Descriptors – convexHull. OpenCV 4.x Documentation. [Online]. Available: https://docs.opencv.org/4.x/d3/dc0/group_imgproc_shape.html [Accessed: Jun. 2026].

- [24] NumPy Developers. `numpy.diff` – NumPy v2.4 Reference. NumPy Documentation. [Online]. Available: <https://numpy.org/doc/2.4/reference/generated/numpy.diff.html> [Accessed: Jun. 2026].

Parte IV

Anexos

9. Manual de Instalación y Uso

A continuación describe de manera detallada los requisitos de software, el proceso de despliegue y los procedimientos necesarios para la correcta ejecución de la aplicación de navegación autónoma. El sistema se ha diseñado bajo una arquitectura de consola para garantizar el determinismo y mitigar cualquier tipo de retardo en el procesamiento.

9.1. Instalación y configuración del entorno

9.1.1. Requisitos del sistema

El software ha sido desarrollado y validado bajo las siguientes especificaciones:

- **Intérprete de Python:** Se requiere estrictamente la versión 3.12.3 de Python ¹. Versiones anteriores podrían no presentar características utilizadas por el programa y versiones más recientes podrían carecer de compilaciones de las librerías de visión.
- **Gestor de paquetes (pip):** Es necesario contar con pip para la gestión de dependencias. En las distribuciones oficiales de Python para entornos Windows y macOS, este gestor se incluye de forma nativa en el instalador. En sistemas basados en GNU/Linux, puede requerir su instalación a través del gestor de paquetes de la distribución (apt, dnf o rpm).

9.1.2. Despliegue del entorno virtual y dependencias

Con el propósito de aislar las librerías del proyecto y evitar conflictos con otras aplicaciones del sistema operativo, se recomienda encarecidamente el uso de un entorno virtual (*virtual environment*). Los pasos para la preparación del entorno desde la carpeta raíz del código fuente del proyecto son:

1. **Creación del entorno virtual:** Ejecutar en el terminal correspondiente el módulo nativo venv:

```
python -m venv venv
```

2. **Activación del entorno:**

- En sistemas de escritorio Windows (PowerShell / CMD):

```
.\venv\Scripts\activate
```

¹Python - <https://www.python.org/downloads/release/python-3123/>

- En sistemas basados en UNIX (Linux / macOS):

```
source venv/bin/activate
```

3. **Instalación de paquetes mediante el archivo de requerimientos:** Una vez activado el entorno (indicado por el prefijo (`venv`) en la línea de comandos), se procede a realizar la instalación de las librerías optimizadas (`opencv-python`, `djitellopy`, `numpy`, entre otras) mediante la orden:

```
pip install -r requirements.txt
```

9.2. Manual de uso y operación

La ejecución del programa se realiza exclusivamente a través de la interfaz de línea de comandos. Esta decisión de diseño es crítica, puesto que la integración de una interfaz gráfica de usuario (*GUI*) tradicional añadiría hilos de ejecución orientados al renderizado visual y dependencias del gestor de ventanas del sistema operativo, introduciendo latencias imprevisibles que podrían comprometer la estabilidad física del dron en pleno vuelo.

9.2.1. Parámetros y argumentos de configuración

El programa expone una serie de variables de entrada gestionados mediante la librería `argparse`. Estos argumentos permiten modificar el comportamiento del software en tiempo de ejecución sin necesidad de modificar el código fuente. A continuación, se presentan los parámetros disponibles:

--test

Tipo: Bandera (Flag booleano) | **Valor por defecto:** False

Activa el modo de simulación o pruebas. Cuando está habilitado, el software inicializa los algoritmos de visión y las matrices de control, pero omite las órdenes físicas críticas hacia los motores del DJI Tello (como la instrucción de despegue `takeoff` o las de movimiento). Permite validar la robustez del código de forma segura en la estación de tierra sin necesidad de comprometer el dron.

--save-video

Tipo: Bandera (Flag booleano) | **Valor por defecto:** False

Determina si la sesión de vuelo actual debe almacenarse en el almacenamiento local de la estación de tierra. Al finalizar la ejecución, el sistema exporta el archivo de vídeo procesado con las superposiciones gráficas de los vectores de error calculados, lo que resulta esencial para las tareas de telemetría y auditoría posterior del control PID.

--timer

Tipo: Bandera (Flag booleano) | **Valor por defecto:** False

Al finalizar la ejecución del programa muestra en la consola un análisis de los

tiempos de ejecución (media, mínimo, máximo y desviación típica) de las principales tareas llevadas a cabo a lo largo del programa.

--calibrate-led

Tipo: Bandera (Flag booleano) | **Valor por defecto:** False

Fuerza al programa a inicializar la subrutina de calibración cromática antes de arrancar el bucle principal de navegación. Es un parámetro crítico y necesario si se opera en laboratorios con condiciones de iluminación desconocidas.

--altitude <int>

Tipo: Entero | **Valor por defecto:** 60

Establece la altura de vuelo en centímetros a la que debe estabilizarse el dron tras el despegue inicial. Este valor modifica directamente el cálculo de la escala métrica del lazo de Roll al alterar la proyección del plano del suelo en el sensor óptico.

--edge-pixels <int>

Tipo: Entero | **Valor por defecto:** 10

Define el umbral de píxeles para la delimitación y filtrado de los bordes del plano de la imagen en los algoritmos de segmentación de la trayectoria.

--calibrate-record <int>

Tipo: Entero | **Valor por defecto:** 3

Tiempo definido en segundos que se utilizará para la grabación de datos del Hue de cada color en la fase de calibración LED.

--calibrate-pause <int>

Tipo: Entero | **Valor por defecto:** 3

Tiempo de espera medido en segundos que habrá de pausa entre la toma de valores entre colores en la fase de calibración LED.

--help

Muestra en la consola un texto de ayuda dinámico con la descripción de todos los parámetros anteriores y finaliza la ejecución del script de manera inmediata.

9.2.2. Ejemplos de Ejecución

Para iniciar la aplicación bajo diferentes escenarios operativos, se pueden utilizar las siguientes configuraciones de comandos en la consola:

- **Ejecución normal de vuelo autónomo con grabación:** Inicia el dron con los valores estables utilizados en el TFG, manteniendo la altura estándar (60 cm) y el guardado de vídeo activo:

```
python main.py --save-video
```

- **Validación de algoritmos en entorno de pruebas:** Útil para realizar depuración de código (*debugging*) en el laboratorio sin activar los rotores del dron, aumentando la anchura escogida para la detección de cortes en los bordes de los fotogramas:

```
python main.py --test --edge-pixels 15
```

- **Operación en entornos de alta luminosidad con calibración lumínica y análisis temporal:** Configura el dron a una altura de seguridad menor (50 cm), inicializa el protocolo de muestreo de color para calibrar los diodos LED de la pista e imprime un análisis temporal de las tareas principales de ejecución al terminar la ejecución del programa:

```
python main.py --calibrate-led --altitude 50 --timer
```

10. Manual Librería djitellopy

10.1. Manual de referencia de la API DJITelloPy y el SDK de Tello

El control y la adquisición de datos del cuadricóptero DJI Tello se realiza mediante el uso de la librería de código abierto DJITelloPy. Esta librería actúa como un *wrapper* o capa de abstracción de alto nivel construida sobre el SDK oficial de Ryze Robotics.

El funcionamiento interno de la arquitectura se basa en la pila de protocolos de internet UDP/IP. La estación de tierra (servidor central) se conecta al punto de acceso Wi-Fi emitido por el UAV (IP por defecto 192.168.10.1) y establece tres canales de comunicación concurrentes e independientes:

- **Canal de Comandos (Puerto UDP 8889):** Envía cadenas de texto plano para ordenar acciones físicas o solicitar telemetría, y recibe confirmaciones de ejecución ("ok" o "error").
- **Canal de Estado (Puerto UDP 8890):** Escucha de forma pasiva una ráfaga continua de datos en formato *string* que detalla las variables dinámicas del dron (giroscopo, acelerómetro, batería, barómetro, etc.).
- **Canal de Vídeo (Puerto UDP 11111):** Recibe el flujo de vídeo en bruto codificado en H.264 procedente de la cámara frontal del dron.

A continuación, se documentan las funciones críticas de la API utilizadas en el desarrollo de este proyecto, detallando su comportamiento lógico, el comando nativo del SDK que invocan y su justificación en el software implementado.

10.1.1. Inicialización y enlace de comunicaciones

Tello()

Descripción: Constructor de la clase principal. Instancia el objeto que encapsula los sockets de red UDP, los hilos de ejecución en segundo plano para la escucha de estados y los buffers de vídeo.

Uso en el proyecto: Se invoca al inicio de `main.py` para inicializar la instancia global del vehículo aéreo.

drone.connect()

Descripción: Establece formalmente el enlace de red con el dron a través del socket de comandos. Bloquea el hilo principal hasta recibir una respuesta afirmativa.

Comando SDK subyacente: "command" (Inicializa el modo SDK del dron).

Uso en el proyecto: Se ejecuta de manera obligatoria antes de iniciar cualquier otra rutina. Si el método falla o agota el tiempo de espera, el programa aborta la secuencia para evitar condiciones de vuelo erróneas.

10.1.2. Gestión del flujo de vídeo

`drone.streamon()`

Descripción: Ordena al firmware del dron activar el chip de captura y comenzar la transmisión de los paquetes de vídeo a través del puerto 11111.

Comando SDK subyacente: "streamon"

Uso en el proyecto: Se invoca inmediatamente después de la conexión para levantar el servicio de streaming de vídeo, necesario para el pipeline de visión (5).

`drone.get_frame_read()`

Descripción: Inicializa un hilo de ejecución independiente encargado de recibir los paquetes de vídeo, decodificarlos en tiempo real y almacenar el último fotograma válido en un buffer accesible. Devuelve un objeto de tipo `BackgroundFrameRead`.

Uso en el proyecto: Proporciona el acceso a la propiedad `.frame`, matriz NumPy de tres canales (RGB) que el script principal consume continuamente para alimentar los algoritmos de segmentación de línea, detección de marcadores y clasificación cromática.

10.1.3. Comandos de acción física y altura

`drone.takeoff()`

Descripción: Ordena el encendido automático de los cuatro motores y el despegue vertical de la aeronave. El dron asciende por defecto y se estabiliza de forma interna a una altura aproximada de entre 70 cm y 80 cm.

Comando SDK subyacente: "takeoff"

Uso en el proyecto: Integrado dentro de la función personalizada `safe_takeoff()` en `control.py`. Es un comando crítico que detiene el flujo lógico durante varios segundos para permitir que los lazos de control internos del Tello estabilicen el vuelo estacionario inicial.

`drone.land()`

Descripción: Inicia la maniobra de descenso controlado hasta detectar el contacto físico con el suelo, momento en el que detiene los rotores por completo.

Comando SDK subyacente: "land"

Uso en el proyecto: Actúa como una rutina de seguridad en `main.py` y `control.py`. Se ejecuta de forma asíncrona ante la pulsación de la tecla de pánico 'k' en el teclado por parte del operador, la detección del marcador ArUco de parada (ID 0), la lectura de la señal LED rosa o tras superar el umbral máximo de reintentos fallidos de despegue.

`drone.get_distance_tof()`

Descripción: Devuelve la distancia vertical hasta el suelo medida en centímetros dada por el sensor ToF (*Time-of-Flight*) que se encuentra en la parte inferior del dron. Tiene un rango operativo óptimo de entre 30 cm y 1000 cm

Comando SDK subyacente: "tof?"

Uso en el proyecto: Se utiliza en `safe_takeoff()` para calcular la discrepancia métrica exacta respecto a la altura deseada (60 cm por defecto) y corregirla; alimenta directamente las ecuaciones trigonométricas (5.5.4) para transformar el error de *Roll* medido en píxeles a unidades métricas reales.

`drone.move_up(cm)` / `drone.move_down(cm)`

Descripción: Ordena una traslación vertical pura hacia arriba o hacia abajo, respectivamente, desplazando al UAV la distancia exacta especificada en el argumento numérico (restringido por el SDK al rango [20, 500] cm).

Comando SDK subyacente: "up x" / "down x" (*x* es el entero en centímetros).

Uso en el proyecto: Empleado en la fase de ajuste de altura en `control.py` posterior al despegue, permitiendo acercar el dron de forma aproximada a la altura deseada.

10.1.4. Guiado continuo

`drone.send_rc_control(roll, pitch, throttle, yaw)`

Descripción: Envía de manera asíncrona un vector de comandos de control remoto equivalente al movimiento de los *joysticks* de una emisora física. Acepta cuatro argumentos enteros normalizados en el rango $[-100, 100]$:

- `roll`: Velocidad lateral (eje geométrico *Y*, positivo derecha).
- `pitch`: Velocidad longitudinal (eje geométrico *X*, positivo adelante).

- **throttle**: Velocidad vertical (eje geométrico Z , positivo arriba).
- **yaw**: Velocidad angular de rotación (eje de guiñada ψ , positivo horario).

Comando SDK subyacente: "rc a b c d" (a, b, c, d corresponden a los canales indicados).

Uso en el proyecto: Es el núcleo de actuación del sistema y se ejecuta al final de cada iteración del bucle principal en `main.py`. Recibe de forma directa los enteros calculados por el regulador PID lateral (**roll**), la velocidad dinámica basado en la solidez del camino (**pitch**) y el regulador PID de orientación (**yaw**), mientras mantiene fijado el canal de aceleración (**throttle**) a cero para delegar la estabilidad en altura al firmware nativo.

Problema de uso: A lo largo del proyecto mediante diferentes pruebas se pudo comprobar que valores bajos (entre un rango aproximado de $[-10, 10]$) de los argumentos de esta función eran menos fiables que valores algo superiores y tendían a no surtir el efecto deseado por ese valor.